Sound Finite Abstraction of Learning-Enabled Discrete-Time Stochastic Hybrid Systems: 2nd Year Review

IMDP Construction, Soundness Certification, and Probabilistic Model Checking under Perfect Perception

Heba Al Kayed

Institute for Perception, Action and Behaviour (IPAB), School of Informatics, University of Edinburgh

Supervisors: Dr Craig Innes and Dr Elizabeth Polgreen

August 2026

Abstract

Stochastic hybrid systems are an established modelling framework for autonomous systems that couple continuous physical dynamics with discrete components, and recent advances in deep learning have produced autonomous systems whose perception of the environment is performed by deep neural networks. Formal verification of such systems is inherently challenging, due to their mixed continuous and discrete dynamics, the uncertainty under which they operate, the dimension of their state and input sets, the size and complexity of the perception networks, and environment conditions that are hard to quantify and are subject to change [13]. This work addresses the abstraction of such systems into finite models on which formal verification techniques such as probabilistic model checking can run. The abstraction carries a guarantee that the finite model provably contains the behaviour of its concrete system, which is what makes verification results obtained on the abstraction meaningful for the system itself. We present a constructive, automatic procedure for generating such abstractions for discrete-time stochastic hybrid systems with learning components. The procedure partitions the state space of the system and generates an Interval Markov Decision Process as the approximation of the original system, in which every transition carries a probability interval certified to contain the corresponding transition probabilities of the concrete dynamics. Our focus is on the probabilistic safety over a finite horizon. We apply the procedure to an Autonomous Emergency Braking System under perfect perception, as the first track of a wider programme on verifying learning-enabled systems and synthesising runtime monitors for them.

Contents

1 Introduction

Cyber-physical systems that interact with uncertain environments, such as autonomous vehicles, robotic systems, and safety-critical control loops, are naturally modelled as *stochastic hybrid systems* [13], in which real-valued physical quantities evolve under stochastic dynamics and are coupled with discrete control or mode variables. Certifying such systems against safety requirements demands quantitative guarantees of the form

> *the probability of an unsafe event within a given time horizon does not exceed a prescribed threshold.*

Probabilistic model checking can answer such questions. Given a model of the system, it computes the probabilities of the events of interest and checks them against the required thresholds [12]. Its algorithms require the model to have a finite, or at most countable, state space, and so do not apply directly to a system whose state space is continuous and therefore uncountable. The system must therefore first be *abstracted* into a model the checker can handle, one whose dynamics approximate the original closely enough that verification results obtained on it carry back to the original system. The construction of such abstractions is itself an established research area with a range of approaches [13]. The present work carries out one such construction, in service of the broader goal of formally verifying stochastic hybrid systems of this kind.

Many of the systems in question are moreover *learning-enabled*, meaning that their perception of the environment is produced by trained components such as deep neural networks (DNNs). Given a concrete, executable model of such a system, this research programme asks whether a finite model can be constructed automatically that

(a) provably preserves quantitative guarantees,

(b) composes with separately quantified perception components, and

(c) supports the offline synthesis of runtime monitors.

This paper answers part (a) for the plant and the controller under perfect perception, and builds the interfaces on which parts (b) and (c) rest.

This work is accordingly positioned among the verification frameworks for learning-enabled autonomous systems, surveyed under the Verified AI agenda of [15]. Compositional verification derives system-level guarantees from component-level guarantees and assumptions between the components [14]. Falsification searches the input space of a closed-loop system for executions that violate a specification, and has been applied to an automatic emergency braking system with DNN perception [7]. Runtime assurance in the Simplex tradition pairs an untrusted controller with a verified fallback and a monitor that switches between them [16]. These frameworks operate on a system-level model that they take as given. The gap this programme addresses is the automatic construction of that model from the concrete dynamics of the system.

The relation to each of these frameworks is complementary. Where falsification searches for violating executions and is silent when it finds none, we compute a certified bracket, a pair of

lower and upper bounds on the violation probability, which is informative precisely when no violation is found. In compositional terms, our abstraction supplies the plant-side component from which a composition rule derives its system-level guarantee. In runtime-assurance terms, the bounds computed offline against our abstraction are what license a monitor's switching decision.

This paper documents a sound finite-state abstraction for an Autonomous Emergency Braking System (AEBS), a representative stochastic hybrid system with three continuous state variables and a small finite action set. Our abstraction procedure produces an Interval Markov Decision Process (IMDP), a finite model whose transitions carry probability intervals rather than exact values. The intervals let a single model hold two kinds of uncertainty at once, the over-approximation introduced when the continuous dynamics are discretised and the perception uncertainty that later stages of the programme will add. A model checker resolves the intervals by robust analysis, computing worst-case and best-case values.

Concretely, we present a pipeline that carries out the abstraction through a sequence of algorithms. The input is the AEBS, given as a discrete-time stochastic hybrid system (dt-SHS, Section 2); the output is emitted as a set of modular PRISM components, together with a certificate that every one-step transition probability of the AEBS lies within the corresponding interval of the model. Under that containment the IMDP is a sound over-approximation of the AEBS and can be fed directly to a probabilistic model checker, here Storm [5], to check the safety properties of interest. For the AEBS these are bounds on the probability of a crash within a given time horizon, evaluated under standard car-to-car lead-vehicle scenarios [9].

The construction is Track 1, the formal-abstraction track of a programme whose end goal is the offline synthesis of deployable runtime monitors. A runtime monitor is a component that operates between the perception components and the controller and guards the system against perception failures. The programme decomposes into three tracks (Figure 1). Track 2 quantifies the uncertainty of the learning-enabled perception components, calibrating them against a test dataset into per-input uncertainty tuples that measure how far the perceived state may depart from the true state. Track 3 composes those tuples with the IMDP of Track 1 through a perception-aware IMDP construction step, and it is from the composed model that the monitor is synthesised.

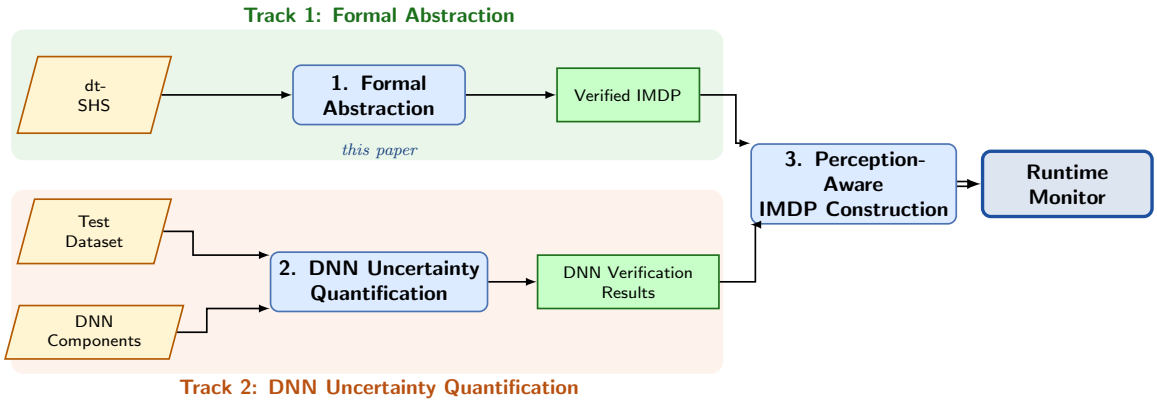


Figure 1: Building blocks of the wider verification programme. This paper documents Track 1.

The remainder of the paper is organised as follows. Section 2 fixes terminology, recalls the probabilistic invariance problem, and states the robust-verification framework on which soundness rests. Section 3 states the requirements the construction must meet and surveys related work against them. Section 4 presents the abstraction procedure and the soundness analysis in detail. Section 5 develops the AEBS case study and reports the certificates and the verification results. Section 7 concludes.

2 Background

This section fixes, in the standard terms of the field, the ingredients on which the abstraction pipeline of Section 4 and the case study of Section 5 rest. These are the class of continuous models we abstract from, the safety question asked of them, the partition approach by which such models are made finite, the interval-valued finite model we abstract to, and the framework by which verification results transfer between the two.

2.1 Discrete-time stochastic hybrid systems

The source models of the abstraction are the stochastic hybrid systems of Section 1, given in discrete time. Following the formalisation of [13, Def. 1.2], a *discrete-time stochastic hybrid system* (dt-SHS) is a tuple

$$\Sigma \;=\; (Q,\; n,\; X,\; U,\; T_x,\; Y,\; h), \tag{1}$$

where

- $Q = \{q_1, \ldots, q_p\}$ is a finite set of discrete modes;
- $n : Q \to \mathbb{N}_{\geq 1}$ assigns to each mode a continuous-state dimension;
- $X \subseteq \bigcup_{q \in Q} \{q\} \times \mathbb{R}^{n(q)}$ is the hybrid state space with Borel σ-algebra $\mathcal{B}(X)$;
- $U \subseteq \mathbb{R}^m$ is a Borel input space;
- $T_x : \mathcal{B}(X) \times X \times U \to [0, 1]$ is a conditional stochastic kernel;
- Y is a Borel output space and $h : X \to Y$ is a measurable output map.

The kernel T_x characterises the one-step transition law. For any $A \in \mathcal{B}(X)$ and any $k \in \mathbb{N}$,

$$\mathbb{P}\big(x(k+1) \in A \,\big|\, x(k), \nu(k)\big) \;=\; \int_A T_x\big(dx(k+1) \,\big|\, x(k), \nu(k)\big), \tag{2}$$

where $x(k) \in X$ is the state at step k and $\nu(k) \in U$ is the input applied at that step. When the kernel admits a conditional density, probabilities under T_x are integrals of that density. The error analyses of the partition-based literature assume this density is Lipschitz continuous, meaning that a bounded change in the state changes the density by at most a proportional amount [13, Def. 3.1].

When $|Q| = 1$ the discrete component is trivial and the model reduces to a discrete-time stochastic control system over a continuous state space [13, Def. 1.3]; this is the form taken by the case study of Section 5.

The safety question asked of such a system is whether it remains within a designated safe region over a finite time horizon. For a dt-SHS Σ with state space X, the *probabilistic invariance* problem [2] is to compute, for each initial state $x_0 \in X$, the probability

$$p_{x_0}(A) \;=\; \mathbb{P}\big(\, x(k) \in A \text{ for all } k \in [0, N] \,\big|\, x(0) = x_0 \,\big), \tag{3}$$

where

- $A \subseteq X$ is the Borel set of safe states;
- N is the finite horizon, counted in steps;
- $x(k)$ is the state at step k under the kernel T_x.

The complementary quantity is the *bounded-reachability* probability

$$r_{x_0}(A) \;=\; 1 - p_{x_0}(A) \;=\; \mathbb{P}\big(\, x(k) \in X \setminus A \text{ for some } k \in [0, N] \,\big|\, x(0) = x_0 \,\big), \tag{4}$$

where $X \setminus A$ is the unsafe set. The safety properties verified in Section 5 are bounded-reachability probabilities of this form. Both quantities are characterised by a backward recursion whose every step integrates against the kernel [2, Thm. 1]; these integrals are generally intractable in closed form, so neither quantity is computable directly on the continuous system.

2.2 Partition-based abstraction

A finite model of a dt-SHS is obtained by partitioning its continuous state space into finitely many cells and treating each cell as one state. On the finite model the backward recursion of Section 2.1 becomes a finite computation. A *partition* of the state space is a finite family of sets satisfying

$$X \;=\; \bigcup_{i=1}^{M} A_i, \qquad A_i \cap A_j = \emptyset \;\text{ for } i \neq j, \tag{5}$$

where

- M is the number of cells;
- $A_1, \ldots, A_M$ are the *cells*, pairwise disjoint measurable subsets of X.

By (2), one step from a continuous state inside a source cell lands in a given target cell with a probability determined by the kernel. This probability generally varies as the state ranges over the source cell. The classical construction ignores the variation: it fixes one *representative point* per cell and adopts the kernel value at that point as the cell-to-cell transition probability [13],

$$P(i,a,j) \;=\; T_x\big(A_j \,\big|\, z_i,\; a\big), \tag{6}$$

where

- $z_i \in A_i$ is the representative point chosen in the source cell A_i;
- $P(i,a,j)$ is the resulting transition probability from cell A_i to cell A_j under input a.

When the input set is finite, the model so produced is a *Markov decision process* (MDP), a finite model with states, actions, and one exact probability per transition [12]. Retaining the within-cell variation instead, as a range of admissible values for each transition, leads to the interval-valued target model of Section 2.3.

2.3 Interval Markov decision processes

The target of the abstraction stores the within-cell variation of Section 2.2 as an interval attached to each transition. Following [10], an *Interval Markov Decision Process* (IMDP, also called a *bounded-parameter MDP*) is a tuple

$$\mathcal{I} \;=\; (S,\; U,\; \check{P},\; \hat{P}), \tag{7}$$

where

- S is a finite set of states and U a finite set of actions;
- $\check{P}, \hat{P} : S \times U \times S \to [0,1]$ are the lower-bound and upper-bound transition functions, with $\check{P}(s,a,s') \leq \hat{P}(s,a,s')$ for every triple.

The IMDP represents the set of all MDPs over S and U whose exact transition probabilities lie within the corresponding bounds and whose transition probabilities out of each state and action sum to one.

For the abstraction of a dt-SHS, the IMDP takes one state per cell of (11) and its actions from the finite input set. The interval attached to the transition from cell A_i to cell A_j under action a is the range of the cell-to-cell probability over the source cell,

$$\check{P}(i,a,j) \;=\; \min_{s \in A_i} T_x\big(A_j \,\big|\, s,a\big), \qquad \hat{P}(i,a,j) \;=\; \max_{s \in A_i} T_x\big(A_j \,\big|\, s,a\big), \tag{8}$$

where the minimum and the maximum are the smallest and the largest value the cell-to-cell probability takes as the state s ranges over the source cell A_i; both are attained under the continuity assumption of Section 4.

2.4 Robust verification of interval models

Verification of an interval model must account for every exact model that its intervals admit. The set of MDPs represented by an IMDP is the object studied as a *Markov set-chain* [11]; a *selection* of the set-chain is a choice, for each state and action, of one transition distribution consistent with the intervals. Properties are expressed in probabilistic computation tree logic (PCTL), the specification language of probabilistic model checking, in which both the invariance and the bounded-reachability probabilities of Section 2.1 are expressible [12]. *Robust verification* of a PCTL property over an IMDP computes the smallest and the largest value, written $P_{\min}$ and $P_{\max}$, that the property attains over the represented set. We call the pair $[P_{\min}, P_{\max}]$ the *bracket* of the property. The value of the property on every MDP of the set, in particular on every selection, lies within the bracket.

The preservation result we rely on is due to [6]. When one model's transition probabilities are contained, transition by transition, within the intervals of the IMDP, the bracket computed on the IMDP contains the property value of the contained model. Containment of the concrete transition probabilities within the abstract intervals is therefore the single hypothesis under which a verification result on the IMDP transfers to the system being abstracted. We make this hypothesis precise and discharge it in Section 4.3. In practice the bracket is computed by robust value iteration, the discrete counterpart of the backward recursion of Section 2.1, resolved at each step against the extreme distributions admitted by the intervals; the model checker Storm implements it for interval models [5].

3 Related Work

This section states what the wider programme needs from prior work and where prior work meets or misses those needs. Two bodies of work matter, the literature on verifying or synthesising autonomous systems whose perception is a deep neural network, the setting of the wider programme, and the literature on abstracting stochastic dynamical systems, the setting of the construction of Section 4.

The nearest neighbour of the wider programme is DeepDECS [3], a method that synthesises discrete-event controllers for autonomous systems with DNN perception. It quantifies the uncertainty of the trained network with confusion matrices, conditioned on the outcomes of DNN verification techniques over a representative test dataset. It injects that uncertainty into a Markov model of the system under perfect perception, and it synthesises controllers that are correct by construction with respect to the system requirements. DeepDECS therefore moves in two steps, a model of the system under perfect perception first and the measured DNN uncertainty added to it afterwards, and the wider programme follows the same two steps (Section 6). The differences are in the task, in the model, and in the uncertainty carried. The programme verifies a system whose controller is given, rather than synthesising the controller, because its target artefacts are runtime monitors shipped alongside the concrete system. Its perfect-perception model is not written by hand as a finite model but abstracted from the continuous dynamics with the containment certificate of Section 4.3, so what is proved about the model holds for the concrete system, and the monitors inherit that transfer. And the uncertainty its intervals carry is not only the aleatory uncertainty of a fixed trained network, uncertainty that cannot be reduced because the network no longer learns, which is the kind DeepDECS quantifies; the same intervals also carry the uncertainty the abstraction itself introduces.

The abstraction the programme needs must meet four requirements.

(i) The target model is interval-valued, an IMDP, so that the noise of the system and the uncertainty of the abstraction are represented natively in the model rather than collapsed to single numbers.

(ii) The abstraction is formally sound, with a certificate that the transition probabilities of the concrete system lie inside those of the model.

(iii) The abstraction is produced at practical cost, in a generation time and file size that allow the model to be handed to a probabilistic model checker.

(iv) The model is open to composition and to later augmentation, so that perception uncertainty measured from the DNNs, and the outputs of further activities such as DNN verification, can be added to the certified system model without rebuilding it.

Systems that mix continuous physical motion with randomness are called stochastic hybrid systems, and their automated verification and synthesis is surveyed by [13]. The survey splits the field into discretisation-based methods, which build finite abstractions, and discretisation-free methods; everything discussed here is on the discretisation-based side. Among its open problems the survey lists the construction of interval Markov models compositionally, from interval models built at the level of the subsystems. A certified interval model of one subsystem, requirement (iv), is the piece of that problem contributed here.

The abstraction of [1] partitions the continuous state space and builds a finite Markov model with one state per cell. The model is accompanied by a bound on the difference between the safety probability computed on it and the safety probability of the concrete system, governed by the regularity of the stochastic kernel and the resolution of the partition. Its target model is point-valued, one number per transition, so requirement (i) is unmet. Our construction keeps its sense of approximate abstraction and obtains soundness from the containment certificate of Section 4.3 instead, under the preservation result for Markov set-chains [6, 11].

Adaptive gridding builds on that route. The procedure of [8] refines the partition adaptively; instead of splitting every cell equally, it splits the cells over which the kernel varies most, measured by a per-transition range bound, its Equation 3.11. Its soundness mechanism is an additive global error bound that grows with the time horizon of the property being verified, and the refinement exists to shrink that bound. In its own experiments the paper reports errors above one, attributes them to deliberately coarse partitions, and states that the bounds converge to zero as the partition is refined, at the cost of longer optimisation times [8]. For the AEBS this mechanism breaks requirement (ii) in two ways. The bound rests on the Lipschitz assumption, a density for the next state that changes at a bounded rate between nearby states; the AEBS kernel (Section 5.2) has no density on its deterministic dimensions, where the cell-to-cell transition probability jumps between 0 and 1, so on those dimensions the bound is not defined. On the one dimension where it is defined, the computed bound exceeds one within a single control period, and a bound above one on a probability excludes nothing. Here refinement does not reduce it; the bound is identical on the two grids of Section 5, because the noise is constant, so the kernel's variation is the same over every cell.

Interval-valued targets meet requirement (i)by construction [13]. An IMDP stores a lower and an upper probability for every transition, so the uncertainty of the abstraction stays in the model instead of accumulating into one error number. The obstacle is requirement (iii): building the model means finding the smallest and largest transition probability over every cell. The survey identifies mixed monotonicity, dynamics enclosed between two order-preserving maps, as what makes those extremes efficiently computable. Our construction takes that route, finding the extremes at cell corners by the corner evaluation of monotone maps of [4], and is developed in Section 4.

Requirements (i) to (iv) taken together are the reason an existing construction was not adopted unchanged, and Section 4 builds one that meets all four.

4 The Abstraction Procedure

This section presents the construction. Section 4.1 partitions the continuous state space into the states of the IMDP; Section 4.2 computes the probability intervals attached to its transitions; Section 4.3 establishes soundness, by the design of the construction and by a per-transition test

against the system's own dynamics. The composition of the per-action rows into the model handed to the verifier is described with the case study (Section 5.3).

The procedure applies to the following class of systems, which mixes deterministic and stochastic state variables; the two kinds of dimension are treated by different mechanisms at every stage of the construction.

Assumption 1 (System class)**.** *The system is a dt-SHS* (1) *with* $|Q| = 1$*, so that the hybrid state reduces to the continuous state* $s \in \mathcal{S}$*, with* $\mathcal{S} \subseteq \mathbb{R}^n$ *bounded and the action set* U *finite. For every action* $a \in U$*:*

(i) Additive independent noise. *One step from state* s *under action* a *lands at*

$$s' \;=\; \mu(s,a) + w, \tag{9}$$

where

- $s' \in \mathbb{R}^n$ *is the successor state;*
- $\mu : \mathcal{S} \times U \to \mathbb{R}^n$ *is the one-step mean map, giving the successor before noise;*
- $w \in \mathbb{R}^n$ *is the noise vector; its components* w_d *are independent of one another. In each* deterministic *dimension,* $w_d = 0$*, written* $\sigma_d = 0$*. In each* stochastic *dimension,* w_d *follows the normal distribution with mean zero and variance* σ_d^2*, written* $w_d \sim \mathcal{N}(0, \sigma_d^2)$*, where the standard deviation* $\sigma_d > 0$ *is constant over* $\mathcal{S}$*.*

Equivalently, the one-step kernel factorises across the dimensions,

$$T_x\big(A_1 \times \cdots \times A_n \mid s, a\big) \;=\; \prod_{d=1}^{n} T_{x,d}\big(A_d \mid s, a\big), \tag{10}$$

where

- T_x *is the one-step kernel of* (2)*;*
- $A_d \subseteq \mathbb{R}$ *is an interval of values for coordinate* d *of the successor state;*
- $A_1 \times \cdots \times A_n$ *is the region of states that meet all* n *of these interval conditions at once, one per coordinate;*
- $T_{x,d}(A_d \mid s, a)$ *is the probability that coordinate* d *of the successor state falls in* A_d*. In a deterministic dimension the coordinate is exactly* $\mu_d(s,a)$*, so this probability is* 1 *if* $\mu_d(s,a)$ *lies in* A_d *and* 0 *if it does not. In a stochastic dimension the coordinate is* $\mu_d(s,a)$ *plus the noise* w_d*, so it is the probability that this noisy value lands in* A_d*.*

(ii) Continuity. *The map* $s \mapsto \mu(s,a)$ *is continuous on* $\mathcal{S}$*.*

(iii) Component-wise monotonicity. *Take any one coordinate of the state and change it upward, keeping the rest of the state as it is. Each coordinate of* $\mu(s,a)$ *then responds in a fixed way: it only ever rises (or stays put), or it only ever falls (or stays put). Which of the two happens depends on the pair of coordinates involved, but never on where in* $\mathcal{S}$ *the change is made.*

The additive form (9) is the standard shape in this abstraction literature [8]; the component-wise monotonicity of part (iii) is the property exploited for corner-based reachable-set computation in [4]; the mixed Dirac and Gaussian split is the restriction this construction is designed for. That the case study satisfies the assumption is established in Section 5; the constancy of σ_d in part (i) is additionally checked at construction time (Algorithm 2).

4.1 Partitioning the state space

Following the partition-based abstraction approach of [1, 8], the continuous state space is covered by a finite grid of non-overlapping cells. Terminology is fixed as follows; Figure 2 depicts the three notions. The space $\mathcal{S}$ belongs entirely to the concrete system, and a *continuous state* is a point of $\mathcal{S}$. A *cell* is a region of $\mathcal{S}$ delimited by the grid. A *state* is an element of the finite model and carries no geometry of its own. The two sides are connected by mapping each cell to one state. The finite model occupying state i means that the concrete system lies at some point of the cell A_i, with no record of which; the transition intervals of Section 4.2 are constructed to be valid for every such point.

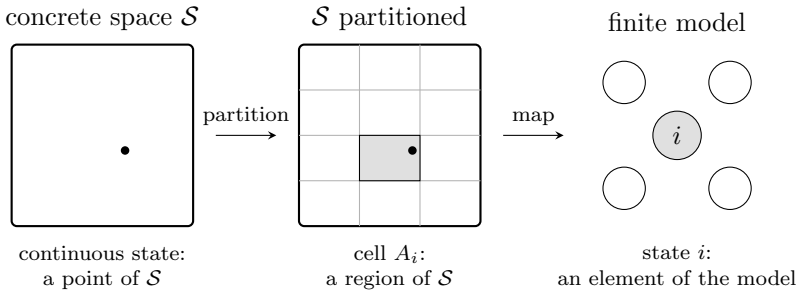


Figure 2: Continuous states, cells, and states. Left: a continuous state is a point of the concrete space $\mathcal{S}$. Centre: the partition delimits cells; the point has not moved and lies in cell A_i. Right: a state is an element of the finite model; cell A_i is mapped to state i.

The complete state set of the model, including its absorbing states, is assembled at the end of this subsection. The bounded state space is covered as

$$\mathcal{S} \subseteq \bigcup_{i=1}^{M} A_i, \qquad A_i \cap A_j = \emptyset \ \text{ for } i \neq j, \tag{11}$$

where

- $\mathcal{S} \subseteq \mathbb{R}^n$ is the continuous state space;
- A_i is the i-th cell, a bounded region of $\mathcal{S}$ containing uncountably many continuous states;
- M is the total number of cells.

The coordinates of $\mathcal{S}$ are indexed $d = 1, \ldots, n$ and called *dimensions*. Each cell is specified by one interval of values per dimension and contains exactly the continuous states whose coordinate in each dimension lies in that dimension's interval; a region of this form is called an *axis-aligned box*, and Figure 3 shows one cell with its defining intervals.
Each interval is *half-open*, containing its lower endpoint and excluding its upper endpoint,

$$A_i \;=\; \prod_{d=1}^{n} \big[\, \mathrm{lo}_{i,d},\ \mathrm{hi}_{i,d} \,\big), \tag{12}$$

where

- $\mathrm{lo}_{i,d}$ and $\mathrm{hi}_{i,d}$ are the lower and upper endpoints of cell i's interval in dimension d.

A point lying on a boundary shared by two cells therefore belongs to exactly one of them, the cell whose lower endpoint it is. This is what makes the cells of (11) pairwise disjoint while covering $\mathcal{S}$: every continuous state lies in exactly one cell.

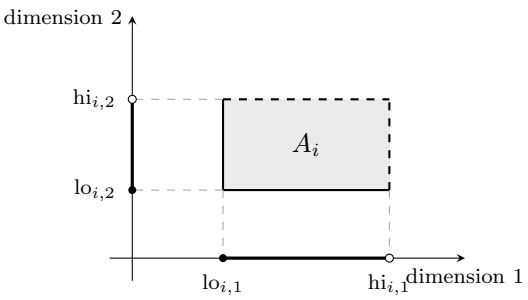


Figure 3: One cell and its defining intervals, drawn for $n = 2$. The cell A_i contains exactly the continuous states whose coordinate in each dimension lies in that dimension's interval. Solid edges belong to the cell and dashed edges do not: each interval contains its lower endpoint (filled) and excludes its upper endpoint (open).

The grid is uniform: in each dimension d, all cells have the same width r_d, the grid *resolution* in that dimension. The cells of a dimension are delimited by a sorted list of boundary values, its *edges*: the first cell lies between the first and second edge, the second cell between the second and third, and so on, as Figure 4 shows. The edges of dimension d are

$$\mathrm{edges}_d \;=\; \Big(\ell_d - \tfrac{r_d}{2},\ \ell_d + \tfrac{r_d}{2},\ \ell_d + \tfrac{3r_d}{2},\ \ldots,\ h_d + \tfrac{r_d}{2}\Big), \tag{13}$$

where

- r_d is the resolution in dimension d;
- ℓ_d and h_d are the lower and upper bounds of dimension d;
- edges_d is the resulting list: it starts half a cell below ℓ_d and advances in steps of r_d until half a cell above h_d.

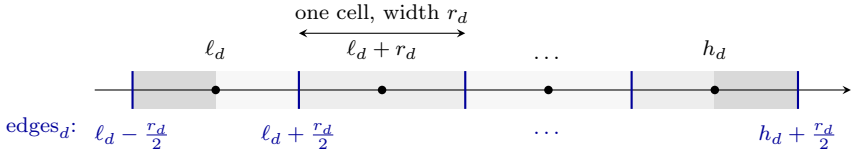


Figure 4: The edges of one dimension, drawn for four cells. The blue ticks are the entries of edges_d in (13); each pair of consecutive edges delimits one cell, shown as alternating strips of width r_d. The black points are the cell centres. The first edge sits half a cell below the lower bound ℓ_d, so the bounds ℓ_d and h_d fall on the centres of the outermost cells; the darker strips are the halves of those cells extending beyond the bounds.

Placing the values $\ell_d,\ \ell_d + r_d,\ \ldots,\ h_d$ in cell interiors rather than on cell boundaries has two effects: floating-point rounding of a value computed at such a continuous state cannot move it across a cell boundary, and the centre of a cell can coincide exactly with the continuous state it represents. Because the outermost cells extend half a cell beyond the bounds, the union of the cells strictly contains $\mathcal{S}$; (11) therefore states a covering rather than an equality.

Writing n_d for the number of cells along dimension d, the grid total cells are described as follows

$$M \;=\; \prod_{d=1}^{n} n_d \tag{14}$$

Each cell is assigned one integer state identifier. The cells are enumerated in *row-major order*, the convention in which the index of the last dimension varies fastest, and the per-dimension cell indices $(k_1, \ldots, k_n)$ are flattened into a single integer. Distinct index tuples receive distinct identifiers, and each identifier corresponds to exactly one cell. The centre c_i of each cell is recorded alongside it. The centre plays no role in the transition construction of Section 4.2, which bounds the dynamics over every continuous state of a cell rather than at any single representative point; it is the evaluation point of the state labelling.

The questions later posed to the model concern regions of the state space, and a *label* is a name attached to states so that such a region becomes addressable. A label is defined by a *predicate*, a yes-or-no condition on the continuous state. The predicate may involve any dimensions and any quantity computed from them, and a state may carry several labels at once. A state receives a label exactly when the label's predicate holds at the centre of the state's cell. A cell that contains continuous states on both sides of a predicate's boundary is therefore labelled by its centre's answer throughout its extent, so what the labels describe is the predicate as quantised on the grid; refining the grid shrinks the cells on which the two disagree.

The model contains *absorbing* states of two kinds. A state is absorbing when, under every action, its only transition is the certain self-loop $[1, 1]$ to itself: a run that enters an absorbing state remains there. The first kind handles a designated *terminal region* of $\mathcal{S}$: the region of a label whose outcome is decided at entry, meaning the property under study is settled the moment a run first satisfies the label's predicate, so the dynamics need not be tracked further. One grid cell inside the region is designated its representative and made absorbing, and the interval construction of Section 4.2 routes to the representative every transition whose reachable set meets the region; a terminal region therefore introduces no state beyond the M grid cells. The construction designates a single terminal region, as the case study of Section 5 has one outcome decided at entry; further such regions would be handled identically, each through its own absorbing representative and label. The second kind is the *safe sink*, one additional state appended after the M grid cells. The dynamics are not confined to the modelled window $\mathcal{S}$, and the sink receives the probability mass of transitions that leave the grid. The kinds carry distinct labels for the verifier: mass absorbed at a terminal representative bears on the property being checked, whereas mass in the sink records departure from the modelled window, and separate labels keep the contributions separately quantifiable.

The construction also records a *partition fingerprint*, a hash of the exact edge values of every dimension; two runs operate on the same partition exactly when their fingerprints match.

Algorithm 1 records this partitioning step: it lays down the cell edges of (13) in each dimension, enumerates the Cartesian product of cell indices in row-major order, attaches the labels at cell centres, designates the terminal-region representative, and appends the safe sink.

Algorithm 1 Grid construction.

Input: bounds (ℓ_d, h_d) and resolution r_d per dimension; labelling predicates; terminal region
Output: partition $\{A_i\}_{i=1}^{M}$, identifiers, centres, fingerprint, labels, designated absorbing states

1: per dimension, compute the cell count and set edges_d (13)
2: enumerate cells; assign row-major identifiers
3: record cell centres and the partition fingerprint
4: attach to each state the labels whose predicates hold at its cell's centre
5: designate the terminal representative; reserve the sink identifier
6: **return** partition, identifiers, centres, fingerprint, labels, absorbing states

4.2 Per-transition probability intervals

Each cell A_i of Section 4.1 corresponds to one state, and both are indexed by i. Transitions of the IMDP connect states, but the one-step law of the concrete system acts on the continuous states inside the cells: for a fixed action $a \in U$ and target cell A_j, the one-step probability

$$T_x(A_j \mid s, a), \qquad s \in A_i, \tag{15}$$

is a function of the source state s and generally takes different values across the cell, while the IMDP carries a single transition from state i to state j. The construction therefore stores, for each non-absorbing source cell A_i, each action $a \in U$, and each reachable target cell A_j, an interval $[p_{\min}, p_{\max}]$ satisfying

$$p_{\min} \;\le\; T_x(A_j \mid s, a) \;\le\; p_{\max} \qquad \text{for every } s \in A_i. \tag{16}$$

This uniform containment over the source cell is the hypothesis under which verification results transfer from the IMDP to the continuous system [6] (Section 4.3). The interval for a transition is built in three steps. First, bound the region that the dynamics can send the source cell to; this bound is the *image box*. Second, for each dimension, bracket the probability that the successor's coordinate lands in the target cell's interval; the deterministic and stochastic dimensions of Assumption (i) are bracketed by separate mechanisms. Third, multiply the per-dimension brackets, which by the factorisation (10) brackets the transition probability itself.

The image box. By Assumption (i), a step from s under a lands at the mean point $\mu(s, a)$ plus the per-dimension noise w. Applying the mean map to every state of the source cell gives the *image set*

$$\mu(A_i, a) \;=\; \{\, \mu(s, a) : s \in A_i \,\}, \tag{17}$$

the set of mean points to which the cell's states are sent (Figure 6). The image set is in general not aligned to the grid and consists of uncountably many points, so it is not enumerated directly. The construction encloses it in the smallest axis-aligned box that contains it, the *image box*

$$B_i^a \;=\; \prod_{d=1}^{n} \big[\, \mu_d^{\mathrm{lo}},\ \mu_d^{\mathrm{hi}} \,\big], \tag{18}$$

where

- B_i^a is the image box of source cell A_i under action a;
- μ_d^{lo} and μ_d^{hi} are the smallest and largest values that coordinate d of the mean $\mu(s, a)$ takes as s ranges over the cell A_i.

The box contains the image set, so any transition bound computed against the box holds for every continuous state of the cell; and it is a product of one-dimensional intervals, which is the form the per-dimension factorisation of (10) requires.

Computing μ_d^{lo} and μ_d^{hi} directly would mean searching over the uncountably many continuous states of the cell. Component-wise monotonicity (Assumption (iii)) removes that search. In one dimension the principle is elementary: a monotone function on an interval attains its smallest and largest values at the interval's two ends. Extending this to a cell requires the cell's counterpart of the interval ends. The cell A_i has one interval per dimension (12); the interval in dimension d runs from $\mathrm{lo}_{i,d}$ to $\mathrm{hi}_{i,d}$, and these two values are its *endpoints*. A *corner* of the cell is a point whose coordinate in every dimension is one of that dimension's two endpoints. Figure 5 shows the four corners of a two-dimensional cell, each labelled by the pair of endpoints that builds it; with two possible endpoints in each of n dimensions, a cell has 2^n corners.

Consider one coordinate of the next-state mean $\mu(s, a)$. By Assumption (iii), raising any one coordinate of the current state s moves it in one fixed direction. To make the chosen coordinate

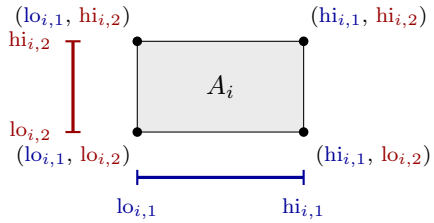


Figure 5: The corners of a cell, for $n = 2$. The cell's interval in dimension 1 (blue, below) has the two endpoints $\mathrm{lo}_{i,1}$ and $\mathrm{hi}_{i,1}$; its interval in dimension 2 (red, left) has the endpoints $\mathrm{lo}_{i,2}$ and $\mathrm{hi}_{i,2}$. Each corner pairs one blue endpoint with one red endpoint, as its label shows; two choices per dimension give $2^n = 4$ corners.

of μ as large as possible over the cell, set each coordinate of s to whichever of its two endpoints moves it upward; the point built from those choices is, by construction, a corner. The smallest value arises in the same way with the opposite choices. Both extreme values are therefore found among the corner evaluations, so scanning all 2^n corners finds them without needing to know which direction any coordinate moves:

$$[\mu_d^{\mathrm{lo}},\ \mu_d^{\mathrm{hi}}] \ = \ \Big[\min_{c \in \mathrm{corners}(A_i)} \mu_d(c, a), \quad \max_{c \in \mathrm{corners}(A_i)} \mu_d(c, a) \Big], \qquad d = 1, \ldots, n, \tag{19}$$

where

- $\mathrm{corners}(A_i)$ is the set of the 2^n corners of cell A_i;
- $\mu_d(c, a)$ is coordinate d of the next-state mean, the mean map evaluated at corner c under action a.

Bounding the image of a box through corner evaluations is the standard construction for monotone systems; the mixed-monotone literature refines it to two evaluations of an auxiliary function [4]. Every transition interval constructed below is computed against this box.

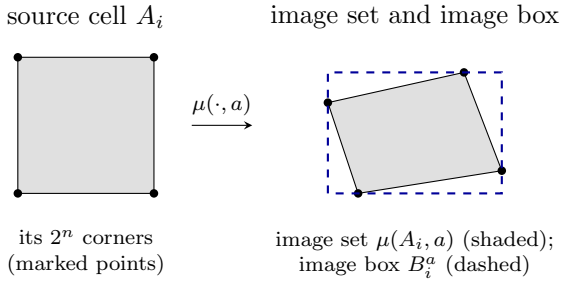


Figure 6: The image box. Left: the source cell, drawn as in Figure 2, with its 2^n corners marked as in Figure 5. Right: the mean map sends the cell to its image set (17); the marked points are the images of the corners. By Assumption (iii) the smallest and largest value of each coordinate over the image is attained at a corner image, so the dashed box they span, the image box (18), encloses the whole image set.

The image box is an intermediate object: it determines the transitions of one source cell under one action and is then discarded, and it is never itself a state of the IMDP. It typically intersects several of the fixed target cells of Section 4.1 (Figure 7), and each target cell the box intersects becomes one transition of the source cell, carrying its own probability interval computed against the box; the cells themselves remain disjoint, it is the box that spans several of them. Because the box is an interval in every dimension, the intersected grid cells form a

contiguous, axis-aligned block in index space: a source cell transitions into a block of target cells, possibly together with the terminal representative and the safe sink, and into no grid target outside the block.

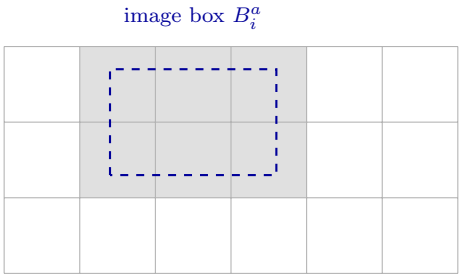


Figure 7: The image box laid over the target grid. The grid is the fixed partition of Figure 2; the dashed box is the image box of Figure 6, in general not aligned to the grid. Each of the six shaded cells it intersects becomes one transition of the source cell; the box is discarded once the transitions are recorded.

A single box encloses the image because the image set is connected: by Assumption (ii) the mean map is continuous for each fixed action, and a continuous map sends the connected source cell to a connected image set. Continuity is required of the dynamics under each fixed action only. The action-selection logic of a controller is typically discontinuous across the state space, and that discontinuity never enters an image-box computation: it is confined to the separate controller component, which only decides which action's transitions apply to each cell (Section 5.3). Generically, if the concrete system has hybrid dynamics, then a hybrid jump could split the image into disjoint pieces, and the construction would then need a union of image boxes per cell; computing per action is what removes that case here.

Per-dimension factor intervals. The factorisation (10) of Assumption (i), instantiated at the target cell, reads

$$T_x(A_j \mid s,a) \;=\; \prod_{d=1}^{n} T_{x,d}\big(\,[\ell_d^j, h_d^j) \,\big|\, s,a\,\big), \tag{20}$$

where

- $[\ell_d^j, h_d^j)$ is the target cell's interval in dimension d, as in (12), so that $A_j = \prod_d [\ell_d^j, h_d^j)$;
- $T_{x,d}$ is the one-dimensional factor of (10).

Each factor on the right of (20) is bracketed by an interval valid for every $s \in A_i$, and the transition interval is the product of the factor brackets, Equation (30). The two kinds of dimension give two kinds of factor bracket.

In a *deterministic* dimension ($\sigma_d = 0$) the factor is the indicator of the event $\mu_d(s,a) \in [\ell_d^j, h_d^j)$: it equals 1 for a continuous state whose mean lands inside the target interval and 0 otherwise. Over the source cell, the mean $\mu_d(s,a)$ ranges exactly over the image interval $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$, so the tightest bracket valid for every continuous state of the cell is

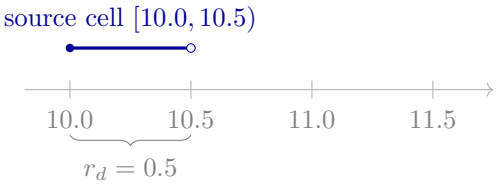
$$\big[\, f_{\mathrm{lo}},\ 1\,\big], \qquad f_{\mathrm{lo}} = \begin{cases} 1 & \text{if } \mu_d^{\mathrm{lo}} \ge \ell_d^j \text{ and } \mu_d^{\mathrm{hi}} < h_d^j, \\ 0 & \text{otherwise,} \end{cases} \tag{21}$$

where

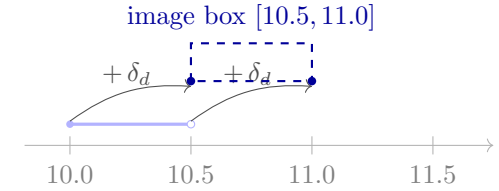
- f_{lo} is the lower bound of the factor bracket; the upper bound is 1 for every enumerated target;

- the first case holds when the image interval lies wholly inside the half-open target interval, the second when it touches or crosses one of its boundaries.

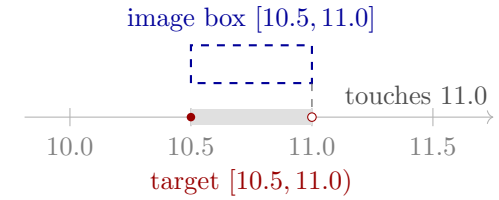
A factor of $[1, 1]$ records a target *certainly* reached in that dimension: the image lies at or above the lower boundary and strictly below the upper one, so the mean of every continuous state lands in the half-open target interval. A factor of $[0, 1]$ records a target only *possibly* reached, and arises in two geometries. If the image *crosses* a boundary of the target interval, some continuous states of the source cell reach the target and others do not, and $[0, 1]$ is the exact range of the indicator over the cell. If the image only *touches* the upper boundary h_d^j, the factor $[0, 1]$ is sound but may be conservative, for the reason given next. Only cells the image interval crosses or touches are enumerated, and each receives the upper bound 1. The upper inequality in (21) is strict because the cells are half-open (Section 4.1): a mean landing exactly on h_d^j belongs to the cell above, not to the target. Touching images arise routinely, not only through rounding: whenever a dimension moves by a fixed amount δ_d equal to the cell width r_d, a source cell aligned with the grid maps exactly onto its neighbouring cell, and the upper endpoint of the image lands precisely on the shared boundary (Figure 8). A non-strict test would record such a touching image as certain, with lower bound 1; yet a continuous state whose mean sits exactly on h_d^j reaches the target with probability 0, so the recorded lower bound would exceed the true probability and violate (16). The strict test records $[0, 1]$ instead, which is sound in every case and conservative only when no state of the half-open source cell attains the boundary.



(a) source cell



(b) corner images span the image box



(c) image box touches the excluded boundary

Figure 8: A touching image from an aligned step. The step $\delta_d = r_d = 0.5$ sends both corners of the source cell onto grid lines, so the image box touches the excluded upper boundary of the target and the factor is $[0, 1]$.

In a *stochastic* dimension ($\sigma_d > 0$) each step splits into a prediction and a landing. The mean map sends the continuous state to a single predicted next value, exactly as in a deterministic

dimension. The landing is then a random draw from a Gaussian density centred on the prediction, with standard deviation σ_d (Figure 9). The prediction is a function of the state alone; the spread around it is the noise. Three terms are kept distinct throughout: the *prediction*, where a density is centred; the *target window*, the half-open interval $[\ell_d^j, h_d^j)$ of a candidate cell; and the *cell middle*, the midpoint of that window. The factor for a target is the landing chance,

$$g(\mu) \;=\; \Phi\Big(\tfrac{h_d^j-\mu}{\sigma_d}\Big) - \Phi\Big(\tfrac{\ell_d^j-\mu}{\sigma_d}\Big), \tag{22}$$

where

- $g(\mu)$ is the chance that the landing falls in the target window $[\ell_d^j, h_d^j)$ when the prediction is μ;
- μ is the prediction, the value the mean map assigns to the continuous state;
- σ_d is the noise level of the dimension, the standard deviation of the Gaussian, constant over the state space;
- Φ is the standard normal cumulative distribution function: $\Phi(x)$ is the chance that a standard Gaussian draw falls below x, a tabulated function available in every numerical library.

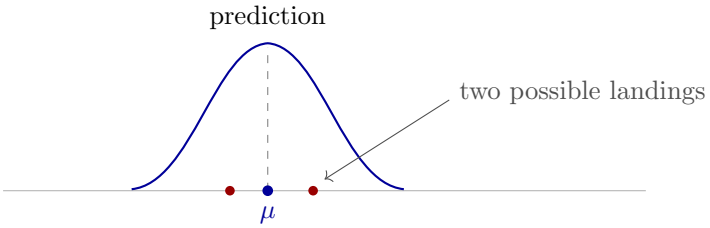


Figure 9: Prediction and landing. The mean map fixes the prediction μ; the density is fixed to it; a landing is a draw from the density. The construction never draws landings: it evaluates their chances in closed form through (22).

A source cell contains a continuum of continuous states, and each state has its own prediction, so the cell owns a family of identical densities whose positions differ. The corner evaluation (19), the same one used for the deterministic dimensions, returns the smallest and largest prediction, and by the monotone continuity of the mean map every value between them is the prediction of some state in the cell. The attainable predictions therefore fill the image interval $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$ exactly, and no prediction outside it is attainable (Figure 10).

The abstraction must attach one factor to the whole cell, valid for every state in it. Different states have different predictions, hence different landing chances, so the factor is the range of the landing chance over the attainable predictions: from the smallest chance attained by any state of the cell to the largest. Both extremes are located in closed form, because the landing chance depends on the prediction in a simple way: it is largest when the prediction equals the cell middle,

$$\mu^\star \;=\; \frac{\ell_d^j + h_d^j}{2}, \tag{23}$$

and it decreases as the prediction moves away from the cell middle in either direction, since the density is symmetric and places less mass on the window the further its centre lies from it. The extremes over the image interval follow immediately. The smallest chance is at the end of the interval farther from the cell middle,

$$m \;=\; \min\big\{g(\mu_d^{\mathrm{lo}}),\; g(\mu_d^{\mathrm{hi}})\big\}, \tag{24}$$

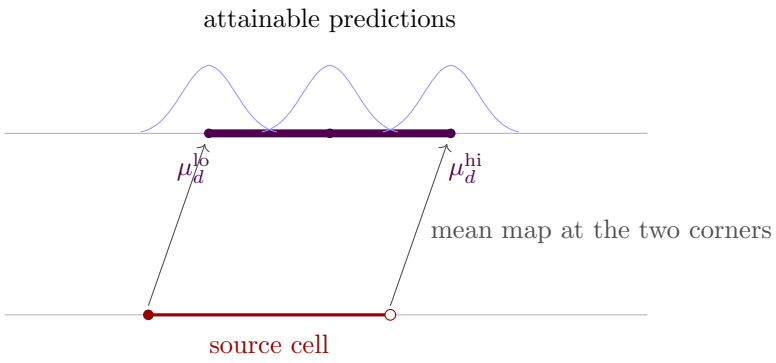


Figure 10: The attainable predictions. The mean map at the two corners of the source cell bounds the predictions; every value between is attained by some state of the cell, and none outside is. Each prediction on the interval carries its own density; three are drawn.

and the largest is at the attainable prediction closest to the cell middle,

$$M \;=\; g(\bar{\mu}), \tag{25}$$

where

- $\bar{\mu}$ is the attainable prediction closest to the cell middle: $\mu^\star$ itself if the image interval contains it, the nearer end of the interval otherwise.

The range $[m, M]$ is the per-transition bound of [8, Eq. 3.11]. Both cases of $\bar{\mu}$ arise for a single source cell, because neighbouring targets have different middles. Figure 11 places two candidate targets against one image interval: the middle of the first lies inside the interval, the middle of the second outside. Figure 12 then computes the bracket for each target in turn.

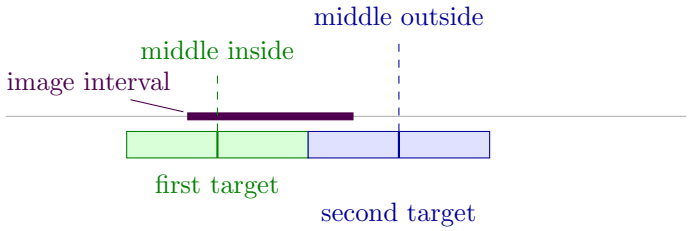


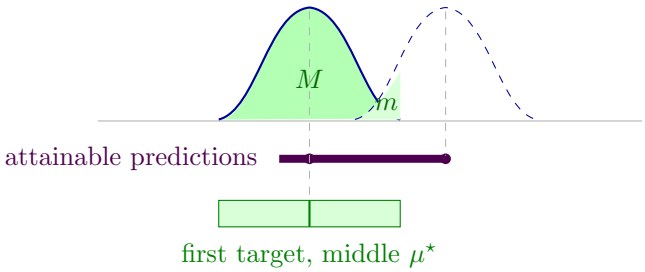
Figure 11: Two candidate targets against one image interval: the first target (green) and the second (blue), each with its middle marked. The first target's middle lies inside the interval; the second's lies outside it. The two targets are carried through Figure 12.

Each candidate target is bracketed independently, over the same image interval; repeating the computation per target gives one bracket per transition. The brackets are bounds, not shares of a fixed total: a target whose window lies away from the image interval receives a bracket small at both ends, with the mass appearing in the transitions to the cells the image covers.

The candidate targets are the cells beneath the image interval extended on each side by the noise reach $n_{\mathrm{cut}}\,\sigma_d$, where n_{cut} is a cutoff parameter. Cells beyond the window are assigned probability zero, an approximation of at most $\Phi(-n_{\mathrm{cut}})$ beyond each end of the window, per transition; n_{cut} is chosen large enough that this mass lies below the numerical tolerance of the containment certificate and the precision of the model checker. The experiments use $n_{\mathrm{cut}} = 6$, for which the ignored mass is of order 10^{-9} (Section 5.3).

State-independence of the noise. The bracket (24)–(25) evaluates g at a single noise magnitude σ_d per source cell. It is therefore valid only under the state-independence of the noise

(a) the first target: middle inside the interval



(b) the second target: middle outside the interval

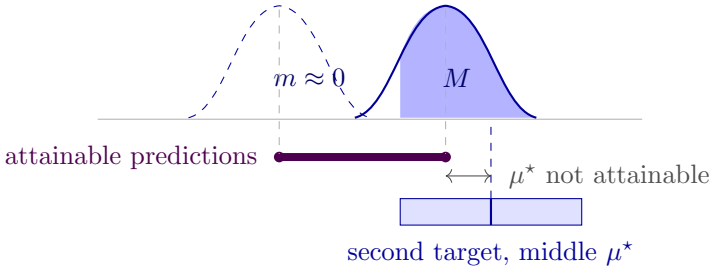


Figure 12: The bracket computed for each target of Figure 11, over the same image interval. Each density stands on its placement, marked by a violet dot on the interval of attainable predictions and tied to the peak by a dashed guide; the shaded area over the target, in the target's colour, is the landing chance. (a) The first target's middle is attainable, so the solid density is placed there, $\bar{\mu} = \mu^\star$, giving the largest chance M (25); the dashed density at the far end gives the smallest, m (24). (b) The second target's middle is not attainable: the solid density is placed at the interval's end, the closest allowed position, and the far-end density does not reach the target, so m is negligible.

required by Assumption (i): were σ_d to vary with the state, g would have to be ranged over the spread of σ_d across the cell as well as over the image. Algorithm 2 checks this part of the assumption at construction time: it records the noise at every corner of the source cell, requires the values to agree to numerical tolerance, and aborts otherwise.

Truncation of the stochastic tail. The grid covers a bounded interval $[b_{\min}, b_{\max}]$ of the stochastic dimension, its *grid band*. The cells of the dimension partition the band: $b_{\min}$ is the lower edge of the first cell and $b_{\max}$ the upper edge of the last. A density whose prediction lies near an end of the band places part of its mass past that end, outside the grid (Figure 13). The abstraction assumes that the dimension remains inside the band, so this mass must be removed from the factors. Each factor is divided by the probability that the landing lies inside the band. Away from the ends of the band this probability is 1 and the factor is unchanged; near an end it is smaller than 1, and the division raises the factor to compensate for the removed mass.

The divisor is the probability that the landing lies inside the band,

$$\Pi(\mu) = \Phi\Big(\tfrac{b_{\max}-\mu}{\sigma_d}\Big) - \Phi\Big(\tfrac{b_{\min}-\mu}{\sigma_d}\Big), \tag{26}$$

where

- $\Pi(\mu)$ is the probability that a landing with prediction μ lies inside the band $[b_{\min}, b_{\max}]$.

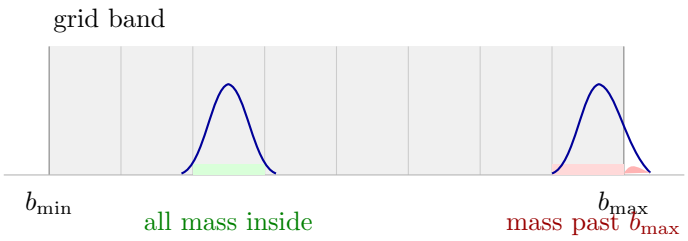


Figure 13: Truncation of the stochastic tail. The cells of the stochastic dimension partition the grid band $[b_{\min}, b_{\max}]$. For an interior cell all mass lands inside the band and the division changes nothing; for a cell at an end of the band, part of the mass falls past the edge and the division compensates for its removal.

For a single continuous state, the stored probability of a target cell is the conditional probability of reaching it given that the landing lies inside the band. This conditional probability is the quotient of the landing chance by the divisor,

$$\frac{g(\mu)}{\Pi(\mu)}, \tag{27}$$

where

- $g(\mu)$ and $\Pi(\mu)$ are evaluated at the prediction μ of that state.

Every value in the image interval is the prediction of some state of the source cell, and each such value has its own divisor. The stored factor must therefore bracket the quotient over the whole image interval. The divisor is largest for a prediction at the midpoint of the band and shrinks toward the band's ends. Over the image interval, the largest divisor is therefore found at the point of the interval closest to the band midpoint, and the smallest at the end of the interval farthest from it. The lower end of the stored factor is divided by the largest divisor and the upper end by the smallest, so that the lower end can only decrease and the upper end only increase:

$$\big[\, m/\Pi_{\text{hi}},\ M/\Pi_{\text{lo}} \,\big], \tag{28}$$

where

- Π_{lo} and Π_{hi} are the smallest and largest values of the divisor over the image interval.

The stored ends are safe for every state of the cell. Each state's conditional probability is built from a landing chance no smaller than m and no larger than M, and a divisor no smaller than Π_{lo} and no larger than Π_{hi}, so it cannot fall below the stored lower end nor exceed the stored upper end:

$$\frac{m}{\Pi_{\text{hi}}} \;\le\; \frac{g(\mu)}{\Pi(\mu)} \;\le\; \frac{M}{\Pi_{\text{lo}}} \qquad \forall \mu \in [\mu_d^{\text{lo}}, \mu_d^{\text{hi}}], \tag{29}$$

where

- $g(\mu)$ and $\Pi(\mu)$ are the landing chance (22) and the divisor (26) at the prediction μ;
- m and M are the ends of the landing-chance bracket (24)–(25), and Π_{lo} and Π_{hi} the smallest and largest divisor over the image interval, as in (28);
- $[\mu_d^{\text{lo}}, \mu_d^{\text{hi}}]$ is the image interval (19).

The two factor types are visible in the generated model. A deterministic dimension has no randomness, so its factor places the whole outcome at one point. The ends of that factor are

therefore 0 or 1, and its value can turn on which cell owns a shared boundary. A stochastic dimension spreads its outcome as a density, and a single point carries no mass under a density, so boundary ownership does not affect its factor, and the factor's ends are fractional. A transition's interval is the product (30) of one factor per dimension, and every fractional entry in the model is the stochastic factor showing through that product (Figure 14).

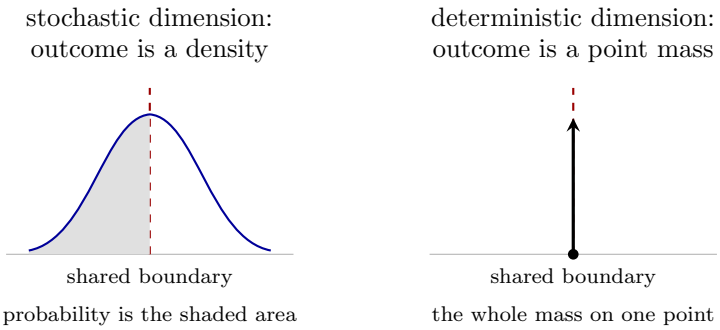


Figure 14: The two factor types. In a stochastic dimension (left) the outcome spreads as a density; a single point carries no mass, so which cell owns a shared boundary does not affect the factor. In a deterministic dimension (right) the outcome is a point mass, drawn on the boundary; which cell owns the boundary decides which cell receives the mass, under the half-open convention of Section 4.1.

Terminal routing. A designated terminal region absorbs the runs that enter it. The routing fills the row of any source cell whose image meets the region, given a membership test that classifies the image as wholly inside, partly inside, or outside the region. When the image is wholly inside, no factors are computed: the row is a single transition to the terminal representative (Section 4.1), with interval set directly to $[1, 1]$. When the image is partly inside, the representative is added as a target beside the grid targets, with interval set directly to $[0, 1]$; the grid targets' intervals are computed by the factor construction as before, and no grid target is enumerated inside the region.

The provided membership test covers a region bounded in a single deterministic dimension: the region lies beyond a boundary value in that dimension, the *terminal boundary*. The dimension is deterministic, so the image box holds the actual next coordinate of every state of the source cell, and membership is read off the image box alone. Each state then either enters the region or does not, so $[0, 1]$ is the exact range for a partly inside image. Grid targets in the bounded dimension are enumerated only up to the terminal boundary. Certain coverage is tested against the whole image, so a grid target straddling the terminal boundary receives the lower bound 0. A richer region, over several dimensions or with a stochastic threshold, requires a richer membership test; the routing itself is unchanged.

Escape in the deterministic dimensions. The grid models a bounded window of each dimension, and the dynamics are not confined to it. Unless floored or capped at the corresponding state-space bound, the image in a deterministic dimension can extend beyond the grid at either end. A state carried past the edge leaves the modelled window, and the safe sink of Section 4.1 is the outcome that records this. Whether an image point has left the grid follows from the half-open convention of Section 4.1. The grid's top edge is the excluded upper edge of the last cell, so a point exactly on it lies outside; the grid's bottom edge is the included lower edge of the first cell, so a point exactly on it lies inside. When the whole image has left the grid, no factors are computed, and the row is a single transition to the sink with interval set directly to $[1, 1]$. When part of the image leaves the grid, the sink is added as a target beside the enumerated grid targets, with interval set directly to $[0, 1]$, the exact range over a cell containing both escaping

and non-escaping states. One grid edge is excepted from the sink routing. In the dimension that carries the terminal region, the grid ends at the terminal boundary, and the states beyond that edge lie inside the terminal region rather than outside the model. An image crossing that edge enters the region and is routed to the terminal representative, as the absorbing definition of Section 4.1 prescribes. The sink routing is required for soundness in the sense of Section 4.3. Mass that leaves the grid must remain representable within its interval row, and a wholly escaped image with no sink target would produce an invalid IMDP.

Product over dimensions. The factor brackets are computed per dimension, and a transition needs one interval for the whole step. For the product kernel of (10), the step succeeds when every dimension lands in its slice of the target, so the brackets multiply, and the interval stored for the target cell with per-dimension indices $(k_1, \ldots, k_n)$ is

$$p_{\min} = \prod_{d=1}^{n} f_{\mathrm{lo}}^{d}, \qquad p_{\max} = \prod_{d=1}^{n} f_{\mathrm{hi}}^{d}, \tag{30}$$

where

- f_{lo}^{d} and f_{hi}^{d} are the ends of the factor bracket of dimension d, evaluated for the target's cell k_d in that dimension;
- n is the number of dimensions.

Each index combination selects one grid cell, and the flattening of Section 4.1 assigns every grid cell a single identifier, so distinct combinations always write to distinct targets. Combinations whose upper product lies below tolerance are discarded; the discarded mass sits below the numerical tolerance of the containment certificate (Section 4.3). A lower product of 0 is stored as 0. For a target only possibly reached, some states of the source cell have true probability 0, and the stored interval must contain that value.

Three procedures carry out the row construction. The first evaluates the corners of a source cell and returns the image box (19). The second brackets the landing chance (22) of a single target window over the image interval, returning the ends (24)–(25). The third calls both and assembles a full row of the IMDP, the transition intervals of the product (30).

Algorithm 2 implements the corner evaluation (19) and checks the noise state-independence required by Assumption (i). The strict test of (21) is exact only at the delimiting edge values, so the corners are evaluated at the stored edges of (13).

Algorithm 2 ImageBox: corner evaluation of the image box, with the noise check.

Input: the stored edges (13) of the source cell; action a
Output: image interval $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$ and noise σ_d per dimension d

1: evaluate the next state and the noise at every corner of the cell, every combination of the stored edges, under a
2: **for all** dimensions d **do**
3: $\quad[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}] \leftarrow$ smallest and largest next value over the corners
4: $\quad$**if** the noise differs across the corners beyond tolerance **then raise** (noise not state-independent)
5: $\quad$**end if**
6: $\quad\sigma_d \leftarrow$ the common noise
7: **end for**
8: **return** the intervals and the noise

Algorithm 3 returns the smallest and largest landing chance over the image interval, the ends of (24) and (25). It applies to the stochastic dimensions only.

Algorithm 3 BoxProbRange: the range of the landing chance over the image interval.

Input: the image interval $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$ of (19); the target window $[\ell_d^j, h_d^j)$ of the candidate cell; the noise level $\sigma_d > 0$

Output: (m, M), the smallest and largest landing chance over the image interval

evaluate the landing chance (22) at both ends of the image interval
$m \leftarrow$ the smaller of the two values, as in (24)
$M \leftarrow$ the landing chance at the attainable prediction closest to the cell middle, as in (25)
return (m, M)

Algorithm 4 assembles one row of the IMDP from the terminal and escape routing, the factor brackets of (21) and (28), and the product (30).

Algorithm 4 RowIntervals: the transition intervals of one source cell under one action.

Input: the stored edges (13) of the source cell; action a; the grid and terminal region of Section 4.1

Output: one row of the IMDP, an interval (16) per reachable target

$[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$ and σ_d, per dimension $d \leftarrow$ ImageBox (Algorithm 2)
if image wholly inside the terminal region **then return** terminal representative, $[1, 1]$
end if
if image wholly outside the grid in a deterministic dimension **then return** sink, $[1, 1]$
end if
for all stochastic dimensions d ($\sigma_d > 0$) **do**
    $\Pi_{\mathrm{lo}}, \Pi_{\mathrm{hi}} \leftarrow$ smallest and largest divisor (26) over $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$
    candidate targets $\leftarrow$ cells beneath $[\mu_d^{\mathrm{lo}}, \mu_d^{\mathrm{hi}}]$ extended by the noise reach $n_{\mathrm{cut}}\sigma_d$
    for all candidate targets k **do**
        $(m, M) \leftarrow$ BoxProbRange on the window of k (Algorithm 3)
        $[f_{\mathrm{lo}}^d,\ f_{\mathrm{hi}}^d]$ of $k \leftarrow$ the truncated bracket (28)
    end for
end for
for all deterministic dimensions d ($\sigma_d = 0$) **do**
    for all cells k the image crosses or touches, up to the terminal boundary **do**
        $[f_{\mathrm{lo}}^d,\ f_{\mathrm{hi}}^d]$ of $k \leftarrow [f_{\mathrm{lo}},\ 1]$ of (21)
    end for
end for
row $\leftarrow$ one transition per factor combination, interval the product (30); drop combinations below tolerance
if image partly inside the terminal region **then** add terminal representative, $[0, 1]$
end if
if image partly outside the grid in a deterministic dimension, terminal edge excepted **then** add sink, $[0, 1]$
end if
return the row

4.3 Soundness of the abstraction

The IMDP stands in for the continuous system at verification time, and the stand-in is sound when every answer computed on it covers the continuous system. The preservation result of [6] (Section 2.4) reduces this to one hypothesis on the construction: the per-transition containment (16), that every stored interval contains the true transition probability of every

continuous state of its source cell. This subsection establishes the hypothesis twice over, by the design of the construction and by a test that recomputes the true probabilities from the system itself.

Containment by design. The corner evaluation (19) returns the exact range of the mean map over the source cell, so the image box encloses the image of every continuous state of the cell. The deterministic factor (21) is exact with respect to the image box under the half-open convention, the stochastic factor brackets the truncated landing chance over the whole image interval (29), and the product (30) of the factor brackets encloses the factorised kernel (10). Every stored interval therefore satisfies (16). Under this containment, every one-step law of the continuous system is a selection of the Markov set-chain that the IMDP denotes, and the verified bracket $[P_{\min}, P_{\max}]$ of Section 2.4 covers the property value of the continuous system [6].

The containment certificate. The certificate compares the abstraction against the system it abstracts, by two independent computations. On one side stands the stored interval, the $[p_{\min}, p_{\max}]$ of (16) that the construction computed from the corners of a source cell. On the other stands the true transition probability at a single continuous state: the deterministic coordinates advanced by the system's own dynamics of Assumption (i), evaluated at that one state, and the stochastic coordinates integrated as the truncated Gaussian of (28). The true side uses no cell, no corner, and no interval of the construction; the two sides share only the definitions both must speak, the boundary convention and the noise truncation.

A cell holds uncountably many continuous states, so the true side is evaluated at a finite sample of them and the design argument above covers the rest. The samples are evenly spaced values in each dimension, from the cell's lower edge to its upper edge, combined across dimensions: s values per dimension give s^n sampled states per cell, and the certified instantiation uses three per dimension, twenty-seven states per cell. The placement respects the half-open convention of Section 4.1. The lowest value lies exactly on the lower edge, which the cell owns; the upper edge does not belong to the cell, so the highest value is the largest floating-point coordinate below it. At each sampled state the test computes the true probability of every reachable target and checks it against the stored interval to a fixed numerical tolerance, recording the excess of any violation. The check runs for every source cell, every action, and every reachable target of the grid. Four further checks run per row. A stored lower bound above tolerance must be met at every sampled state, including a state whose one-step law does not reach the target. A row must contain at least one transition. The row's interval ends must admit a probability distribution, the upper ends summing to at least one and the lower ends to at most one. A sampled state whose image in a deterministic dimension has left the grid must find the sink transition in the stored row. A run of the test over the whole grid is the certificate of one configuration.

Scope of the soundness claim

The claim covers the modelled system, not the raw physics. Any cross-dimension correlation the raw physics carries is removed in adopting the factorised kernel (10), and the removal is a modelling choice made before the abstraction begins. Both sides of the certificate apply this choice, so the certificate relates the IMDP to the system as modelled.

4.4 Choosing the partition resolution

The resolution r_d of (13) is the width of the cells in dimension d, and choosing the resolutions decides how finely the partition divides the state space. The choice matters because a cell maps all of its continuous states to a single state of the model (Section 4.1), and the transition intervals of a row must hold for every continuous state of the source cell. Continuous states grouped in one cell are indistinguishable to the model, so a resolution too coarse for the dynamics collapses continuous states whose behaviour genuinely differs into one cell and hides their differences inside

widened intervals. A finer resolution keeps distinct behaviour in distinct cells and narrows the intervals, at the price of more cells. Model checking returns meaningful answers only when the resolutions preserve the distinctions the question depends on. This subsection describes what a resolution preserves, what it hides, and the considerations by which one is chosen.

Per-step drift. The first consideration is how far the model can stray from the modelled dynamics in a single step. In a deterministic dimension, a target cell partly covered by the image box carries the lower bound 0 (21) and a positive upper bound, and the intervals of a row admit every probability distribution that stays within them (Section 2.4). An admitted distribution may therefore place its mass in any target the image box touches, up to r_d away from where the modelled dynamics put it. The stray renews at every step, because the intervals constrain each step separately and nothing ties one step's distribution to the next. The set of states the model can reach therefore grows by up to r_d per step in each deterministic dimension d, wherever earlier mass landed. This renewed-per-step growth is the wrapping effect of interval analysis [WRAPPING-EFFECT REFERENCE].

The crossing horizon. A model built on a coarse grid can reach the terminal region (Section 4.1) sooner than the system it abstracts, because every step carries drift as well as the modelled motion. Suppose the terminal region is reached by moving along one deterministic dimension, the *approach dimension*. In a single step the model moves towards the terminal region by at most the largest movement the modelled dynamics allow plus one cell width of drift. Dividing the distance from the start state to the terminal region by this largest per-step movement estimates the step at which the model first reaches the terminal region, the *crossing horizon*,

$$k_{\text{cross}} \;\approx\; \frac{D}{\delta^{\max} \;+\; r}, \tag{31}$$

where

- k_{cross} is the crossing horizon, the estimated number of steps until the model first reaches the terminal region;
- D is the distance from the start state to the terminal region, measured along the approach dimension;
- $\delta^{\max}$ is the largest movement towards the terminal region that the modelled dynamics allow in one step;
- r is the resolution of the approach dimension, adding one cell width of drift each step.

Figure 15 draws the estimate. Below the crossing horizon no behaviour the model admits reaches the terminal region. Above it, admitted behaviours reach the terminal region whether or not the modelled dynamics do.

Control visibility. An action is useful only if the grid registers it. The model sees an action through the target cells its image box touches. One step under action a displaces the image box by some amount $\delta_d(a)$ in each dimension d. A displacement smaller than r_d in every dimension is absorbed by the per-step drift, and the model cannot separate taking the action from not taking it. An action is expressible on the grid only if $r_d < |\delta_d(a)|$ in some dimension d.

Resolution against the noise. In a stochastic dimension the resolution is measured against the noise. The stochastic factor of (24) and (25) brackets the landing chance, the probability that the noisy coordinate lands in the target cell, over every position of the prediction in the image interval (Section 4.2). When the cell width r_d grows large against the noise spread σ_d, the

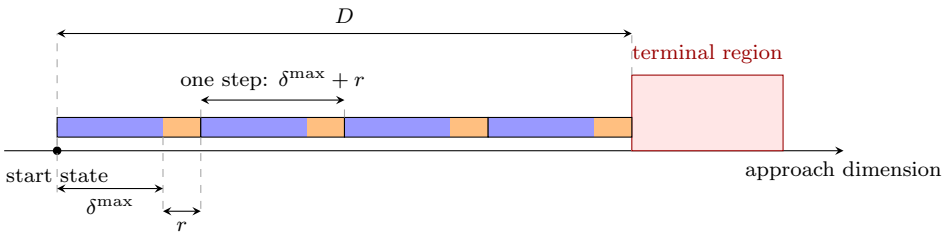


Figure 15: The crossing horizon, drawn for $k_{\text{cross}} = 4$. The start state lies at distance D from the terminal region along the approach dimension. Each step moves the model by at most $\delta^{\max}$ from the modelled dynamics (blue) plus r of drift (orange). The fourth step reaches the terminal region.

landing chance varies from near 1 to near 0 as the prediction moves by one cell width; Figure 16 draws the landing chance against the position of the prediction for the two cases. The bracket then widens towards $[0, 1]$, which admits every probability and rules nothing out. Keeping the ratio σ_d / r_d at order one or above keeps the bracket informative.

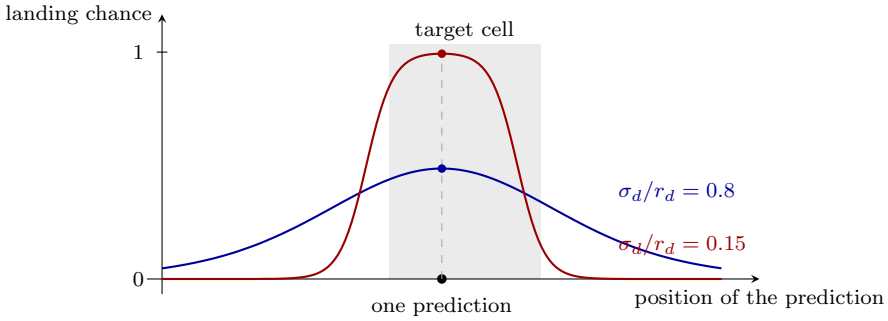


Figure 16: The landing chance in one target cell (shaded) as the prediction moves, drawn for two ratios of noise spread to cell width. Each point of a curve gives the landing chance for a prediction at that position; one prediction at the middle of the target cell is marked, with its landing chance on each curve. With the cell width comparable to the noise spread (blue), the landing chance stays moderate everywhere. With the cell width large against the noise spread (red), the landing chance is near 1 for a prediction inside the target cell and near 0 one cell width away.

The cell budget. Finer resolutions cost cells. The cell count M of (14) multiplies across dimensions, and memory and disk cost grow with M, so the affordable count is capped by the generation infrastructure. Within the cap, a finer resolution in one dimension is afforded by coarsening another dimension or by shrinking the modelled window.

5 Case Study: An Autonomous Emergency Braking System

This section applies the construction of Section 4 to an Autonomous Emergency Braking System (AEBS), a system responsible for braking when there is a potential for collision. The system consists of three components, a perception component that perceives the environment the vehicle is in, a controller that relies on the perception component to issue braking commands, and a plant that executes the commands the controller issues, as Figure 17 shows.

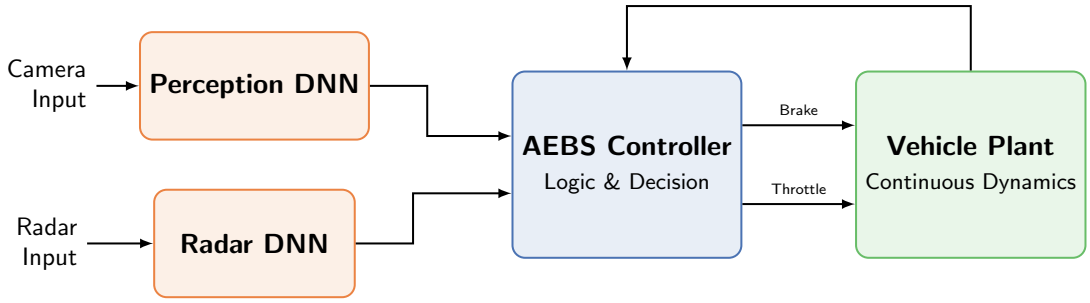


Figure 17: The AEBS and its interfaces.

The IMDPs that the abstraction produces are sound by the containment certificate of Section 4.3. We feed the abstracted system into a probabilistic model checker, the verifier, to understand its behaviour with respect to a safety property, here a crash safety property. The IMDPs are translated automatically into the input language of the verifier and checked for the worst-case and best-case probability of a crash within a bounded time, from named start states and from whole labelled regions. We generate a coarser grid and verify that the worst-case crash probability is certified to be zero up to a horizon that depends on the start state. We then generate a finer grid, restricted to a car-following window. On both grids the best-case probability is zero from every start state that is not already certain to crash. Section 5.1 describes the system, Section 5.2 the modelling choices, Section 5.3 the abstractions and their certificates, Section 5.4 the verification results, and Section 6 what the results mean for the wider programme.

5.1 System components

In each *control period*, the fixed time between successive controller decisions, the perception layer hands the controller the current state, the controller commands an acceleration, and the plant advances the state under that acceleration. The perception layer has a camera-based and a radar-based component. Under perfect perception the perception layer passes the state through unchanged, so the controller receives the true state, and both the perception layer and the controller are translated exactly into the verifier's input language, while the plant alone is abstracted (Section 5.3).

The plant. The plant carries the continuous state

$$s \;=\; (g,\, v_{\text{ego}},\, v_{\text{lead}}) \;\in\; \mathcal{S} \;=\; [0, 100] \times [0, 30] \times [-1, 31] \;\subset\; \mathbb{R}^3, \tag{32}$$

where

- g is the gap from the ego vehicle to the lead vehicle (m), a deterministic dimension;
- v_{ego} is the ego velocity (m/s), a deterministic dimension;
- v_{lead} is the lead velocity (m/s), the stochastic dimension of Assumption (i).

The bounds are modelling choices. The lead velocity is physically nonnegative; its range extends below zero so that the noise of Section 5.2 around a stationary lead stays inside the grid.

The controller. The controller commands one of three accelerations,

$$U \;=\; \{\, a_0 = 0,\; a_1 = -4,\; a_2 = -9.8 \,\}\ \text{m/s}^2, \tag{33}$$

where a_0, a_1, and a_2 are named *coast*, *brake*, and *emergency*. It decides through two quantities derived from the state,

$$v_{\text{rel}} \;=\; v_{\text{ego}} - v_{\text{lead}}, \qquad \text{TTC} \;=\; \begin{cases} g / v_{\text{rel}} & v_{\text{rel}} > 0.1, \\ \infty & \text{otherwise}, \end{cases} \tag{34}$$

where

- v_{rel} is the closing speed (m/s);
- TTC is the time to collision (s), unbounded below a closing speed of 0.1 m/s.

The decision is

$$a \;=\; \begin{cases} a_2 & \text{if } v_{\text{rel}} < 5 \text{ and } g < 2, \quad \text{or } v_{\text{rel}} \geq 5 \text{ and TTC} < 1.0, \\ a_1 & \text{if } v_{\text{rel}} < 5 \text{ and } 2 \leq g < 6, \quad \text{or } v_{\text{rel}} \geq 5 \text{ and } 1.0 \leq \text{TTC} < 1.6, \\ a_0 & \text{otherwise,} \end{cases} \tag{35}$$

where g is in metres, v_{rel} in m/s, and TTC in seconds. The thresholds and decelerations are taken from the Euro NCAP AEB car-to-car test protocol [9]. The controller is deterministic and memoryless, deciding from the current state alone.

Formal classification. The abstraction of Section 4 is defined for a discrete-time stochastic control system [8, Sec. 3]. Its state moves in a continuous space, in discrete time steps, under a finite set of inputs, with the next state drawn from a distribution that depends on the current state and input,

$$\Sigma \;=\; (\mathcal{X},\, U,\, T_x), \tag{36}$$

where

- $\mathcal{X}$ is the continuous state space;
- U is the finite set of inputs;
- T_x is the transition kernel, giving for each state and input the distribution of the next state.

Table 1 instantiates the triple for the AEBS.

Component	In (36)	For the AEBS
State space	$\mathcal{X}$, continuous	$\mathcal{S}$ of (32)
Inputs	U, finite	U of (33)
Kernel	T_x	the kernel (44) of Section 5.2

Table 1: The AEBS as a discrete-time stochastic control system.

5.2 Modelling choices

This subsection fixes the model of the plant. It gives the one-step dynamics, the noise, the transition kernel they define, the check that the assumptions of Section 4 hold, the labels and the terminal region, and the two grids on which the plant is abstracted.

One-step dynamics. Given the state s and the commanded acceleration a, the deterministic part of the next state is

$$a_{\text{ego}} \;=\; \alpha \cdot a, \tag{37}$$
$$v'_{\text{ego}} \;=\; \max\{0,\; v_{\text{ego}} + a_{\text{ego}} \cdot \Delta t\}, \tag{38}$$
$$\bar{v}'_{\text{lead}} \;=\; \max\{0,\; v_{\text{lead}}\}, \tag{39}$$
$$g' \;=\; g + \left(\bar{v}'_{\text{lead}} - v'_{\text{ego}}\right) \cdot \Delta t, \tag{40}$$

where

- the prime marks values at the next step;
- $\Delta t = 0.1\,\mathrm{s}$ is the duration of one control period, and one step of the model of Section 4 is one control period;
- $\alpha = 0.5$ is a static actuator gain, a modelling choice standing in for the delay with which the vehicle's actuators respond;
- $\bar{v}'_{\mathrm{lead}}$ is the lead's predicted next velocity, its current velocity floored at zero, that is, a lead predicted to hold its speed; the lead's motion is an input to the model, not a controlled quantity, and no controller is assumed for the lead.

Noise. The lead's speed changes randomly from one control period to the next, and this is the only randomness in the plant.

Assumption 2 (Lead-vehicle uncertainty)**.** *In each control period the lead vehicle's acceleration is a Gaussian random variable with mean zero and standard deviation σ_a, drawn independently of every other control period. Given the state, the commanded acceleration, and that draw, the next state is determined.*

The noise constants are

$$\sigma_a \;=\; \sqrt{\sigma_{\mathrm{plant}}^2 + \sigma_{\mathrm{lead}}^2} \;=\; \sqrt{0.05^2 + 2.0^2} \;\approx\; 2.0006\ \mathrm{m/s^2}, \tag{41}$$

$$\sigma_v \;=\; \sigma_a \cdot \Delta t \;\approx\; 0.2001\ \mathrm{m/s}, \qquad v'_{\mathrm{lead}} \;\sim\; \mathcal{N}\big(\bar{v}'_{\mathrm{lead}},\, \sigma_v^2\big), \tag{42}$$

where

- $\sigma_{\mathrm{lead}} = 2.0\,\mathrm{m/s^2}$ is the lead-acceleration noise, chosen so that the standard deviation of the lead's speed change per control period is $0.2\,\mathrm{m/s}$;
- $\sigma_{\mathrm{plant}} = 0.05\,\mathrm{m/s^2}$ is a small allowance for unmodelled plant disturbance;
- σ_v is the resulting standard deviation of the lead velocity after one control period, and v'_{lead} is the lead velocity at the next step, distributed around the prediction $\bar{v}'_{\mathrm{lead}}$ of (39).

Both values are modelling choices, not calibrated against measured driving data.

The gap noise. The lead velocity enters the gap update, so the next gap inherits noise of standard deviation

$$\sigma_g \;=\; \sigma_v \cdot \Delta t \;\approx\; 0.02\ \mathrm{m}. \tag{43}$$

where σ_g is the standard deviation of the next gap. If the next gap were computed from the realised lead velocity, the gap and the lead velocity would be jointly random, and the kernel would no longer split into one factor per dimension as Assumption (i) requires. Equation (40) therefore computes the next gap from the predicted lead velocity, which makes the gap deterministic; the noise this drops is $0.02\,\mathrm{m}$ per control period, well below every gap resolution used (Table 3).

The kernel. The next state is

$$s' \;=\; (g',\; v'_{\mathrm{ego}},\; v'_{\mathrm{lead}}), \tag{44}$$

where

- s' is the next state, and its distribution given the state s and the action a is the transition kernel T_x of (36);

- g' and v'_{ego} are the deterministic values of (38) and (40);
- v'_{lead} is the lead velocity drawn from the Gaussian of (42) around the prediction $\bar{v}'_{\text{lead}}$.

All the randomness of T_x is in v'_{lead}, so the probability of moving from s under a into a cell is the probability that v'_{lead} falls in that cell's v_{lead} range if the cell contains (g', v'_{ego}), and zero otherwise. This is the factorised kernel of Assumption (i), deterministic in g and v_{ego} and Gaussian in v_{lead}, and it is the kernel against which the containment certificate of Section 4.3 is evaluated. The one-step map (38)–(40) is continuous and monotone in each state coordinate, as Assumptions (ii) and (iii) require, since it composes linear maps with the floor at zero, $\max\{0, x\}$.

Labels and terminal region. The labels of Section 4.1, names attached to states by predicates on the continuous state, are here predicates over g and TTC of (34), evaluated at cell centres; Table 2 lists them. The `warning` and `critical` predicates use the warning and emergency thresholds of [9], each triggered by distance or by time to collision. The regions they define are fixed by these thresholds alone; they are not the states in which the controller (35) brakes. The terminal region of Section 4.1 is the region of `crash`, the *crash region*, whose outcome is decided at entry.

Label	Predicate at the cell centre
`crash`	$g \leq 0$
`warning`	$g < 10\,\text{m}$ or $\text{TTC} < 2.6\,\text{s}$
`critical`	$g < 2\,\text{m}$ or $\text{TTC} < 1.0\,\text{s}$
`clear`	$g > 0$ and not `warning`

Table 2: The labels used in the results.

The grids. The plant is abstracted twice, on the two grids of Table 3, named after their resolution and their role. The medium production grid covers the whole state space and carries the scenarios of Table 6. The extra-fine Following grid covers a car-following window at resolution 0.1 in every dimension and carries the car-following scenarios of Table 9. Mass that leaves a grid is routed to the safe sink of Section 4.1; mass beyond the v_{lead} range is truncated and renormalised within the range (28).

	Medium production grid	Extra-fine Following grid
Bounds $(g, v_{\text{ego}}, v_{\text{lead}})$	$[0, 100] \times [0, 30] \times [-1, 31]$	$[0, 16] \times [4, 16] \times [5, 15]$
Resolution $(r_g, r_{v_{\text{ego}}}, r_{v_{\text{lead}}})$	(1.0, 0.5, 0.1)	(0.1, 0.1, 0.1)
Cells per dimension $(n_g, n_{v_{\text{ego}}}, n_{v_{\text{lead}}})$	(101, 61, 321)	(161, 121, 101)
Cells M	1,977,681	1,967,581

Table 3: The two grids. Kernel, controller, lead prediction, and truncation policy are identical on both.

Control visibility means that one control period of braking must move the ego velocity by more than one cell, or the grid cannot see the braking; the noise is resolved when its standard deviation per control period is at least one cell wide. Table 4 sets the ego-velocity change of each braking action, produced through the gain α of (37), and the lead-velocity noise of (42) against the cell widths of the two grids. The medium production grid sees neither braking action and the extra-fine Following grid sees both; both grids resolve the noise.

Consideration	Per control period (m/s)	Cell width (m/s)	
		Medium production	Extra-fine Following
Brake, ego velocity	0.2	0.5	0.1
Emergency, ego velocity	0.49	0.5	0.1
Noise, lead velocity	0.2	0.1	0.1

Table 4: The considerations of Section 4.4 on the two grids, each change or noise per control period set against the cell width of the dimension it moves.

The crossing horizon (31) instantiates for the AEBS as

$$k_{\text{cross}} \approx \frac{g_0}{v_{\text{ego}} \cdot \Delta t + r_g}, \tag{45}$$

where

- g_0 is the starting gap, how far the ego starts from a crash;
- $v_{\text{ego}} \cdot \Delta t$ is the most the gap can physically shrink in one control period from that state, since the lead is never predicted to reverse and no action speeds the ego up;
- r_g is the gap resolution, the one cell of drift per control period that the intervals allow (Section 4.4).

5.3 Abstraction results

This subsection reports what the abstraction produced on the two grids of Table 3. It gives the IMDPs, their soundness certificates, and the form in which they are handed to the verifier together with what the verifier is asked.

The IMDPs. Table 5 summarises the two IMDPs. Both were generated on a departmental cluster node with 64 cores and 503 GB of memory, the production grid parallelised over 63 worker processes.

	Medium production grid	Extra-fine Following grid
States (the grid cells and one safe sink)	1,977,682	1,967,582
Transitions	307,323,579	311,491,148
Generation time	approximately 43 min	—
Size of the file handed to the verifier	10.2 GiB	10.4 GiB

Table 5: The two IMDPs. A dash marks a quantity not recorded.

Soundness certificates. The containment test of Section 4.3 was run over every cell of both grids under all three actions, sampling 27 states per cell, and counting a probability as outside its interval only if it fell outside by more than 10^{-7}, an allowance for floating-point error. Neither run found a violation, so by Section 4.3 both IMDPs are certified sound over-approximations of the kernel (44).

Handing the IMDPs to the verifier. The verifier is Storm [5]. A model of this size, about two million states and three hundred million transitions, exceeded the memory of the cluster node when written in the PRISM language and parsed. Each IMDP is therefore written in Storm's explicit format (DRN), a direct list of states, transitions, and labels. For the medium production grid the PRISM file was 28.9 GiB and the explicit file 10.2 GiB. The explicit file carries the closed loop, one action per state, the controller's decision (35) at the centre of that state's cell. For

each query the verifier computes two numbers. For $P_{\max}$ it plays the adversary, choosing at every step, among the transition probabilities the intervals admit, those that make a crash most likely; for $P_{\min}$ it chooses those that make a crash least likely. The pair $[P_{\min}, P_{\max}]$ is the bracket of Section 2.4, and the crash probability of the modelled system lies inside it (Section 4.3). From every start state not already certain to crash, the lowest crash probability the model allows is 0, because the one cell of freedom per control period that the intervals give the verifier, the drift of Section 4.4, is enough to avoid the crash region altogether. Every bracket therefore carries its information in $P_{\max}$, and the results report $P_{\max}$.

5.4 Verification results

The certified IMDPs are checked for the probability of a crash within a bounded number of control periods, from named start states and from the labelled regions of Table 2. This subsection gives the query, the results on the medium production grid, the measurements that fixed the extra-fine resolution, the results on the extra-fine Following grid with a simulation that corroborates them, and the cost.

The query. Every query has the form

```
F<=k "crash",
```

where k is the horizon, the number of control periods within which a crash is asked about. Storm 1.13.0 reads the file as an interval MDP and applies robust value iteration [5]; each query is one pass over the whole model, and the start state or region it reports is selected by its label.

Results on the medium production grid. Table 6 lists the start states queried on the medium production grid, four along a rear-end approach to a stationary lead, the Car-to-Car Rear stationary scenario of [9], and one inside the crash region.

Scenario	Start state $(g, v_{\text{ego}}, v_{\text{lead}})$
Safe Cruising	(100, 15, 0)
Warning Brake	(30, 7, 0)
Emergency Brake	(10, 10, 0)
Imminent Collision	(5, 30, 0)
Post Collision	(0, 0, 0), inside the crash region

Table 6: Scenarios on the medium production grid.

At horizon 100, ten seconds, Post Collision and Imminent Collision return $[1, 1]$, a start inside the crash region and a start that cannot stop in time. Every other scenario and every region of Table 2 returns the trivial bracket $[0, 1]$, which rules nothing out. The intervals allow one gap cell of drift per control period, and ten seconds of drift cover the whole gap range (Section 4.4). The same grid is queried at the horizons

$$k \in \{5,\ 10,\ 15,\ 20,\ 25,\ 30,\ 40,\ 50,\ 75\}, \tag{46}$$

where k ranges over the queried horizons, five control periods apart up to 30 and wider beyond; Table 7 gives the result. For every scenario $P_{\max}$ is 0 up to one queried horizon and 1 from the next, so no fractional value appears between the horizons queried, and $P_{\min}$ is 0 for these scenarios at every horizon. The horizons up to which the worst-case crash probability is certified zero are ordered by starting gap.

Scenario	$P_{\max} = 0$ up to	$P_{\max} = 1$ from
Emergency Brake	$k = 5$ (0.5 s)	$k = 10$ (1.0 s)
Warning Brake	$k = 15$ (1.5 s)	$k = 20$ (2.0 s)
Safe Cruising	$k = 30$ (3.0 s)	$k = 40$ (4.0 s)

Table 7: Worst-case crash probability on the medium production grid at the horizons of (46). No horizon between the two columns was queried. $P_{\min}$ is 0 for these scenarios at every horizon. Imminent Collision and Post Collision are omitted.

Fixing the extra-fine resolution. The medium production grid locates the horizon at which each result flips from 0 to 1 and returns no value in between. Finding the resolution that does return values in between took several experiments on a reduced window, each halving the cells of one dimension while holding the others at the medium production resolution, to find the dimension that needs the most granularity. That dimension is the ego velocity. One control period of emergency braking changes the ego velocity by 0.49 m/s (Table 4), so with ego cells of 0.5 m/s the grid cannot see the braking. Halving the ego cells to 0.25 m/s let it see the braking and returned a fractional worst-case crash probability, 0.999865419 for Safe Cruising, at a horizon where the 0.5 m/s cells already return 1. The extra-fine Following grid therefore uses 0.1 in every dimension, finer than both braking changes of Table 4.

The crossing horizon (45) predicts the horizon at which $P_{\max}$ first leaves 0; the measured crossing is the first queried horizon at which it does. The prediction ignores the controller's braking, so it is an estimate rather than a bound, and it agrees with the measurement. With gap cells of 1.0 m it predicts horizon 40 and the measured crossing is 40; with gap cells of 0.5 m it predicts 50 and the measured crossing is 50.

Results on the extra-fine Following grid. Two things are needed for a fractional worst-case crash probability. The grid must be fine enough to see the braking and to keep the drift small, which the extra-fine resolution provides. And the start state must be one whose outcome is not decided by the physics alone, since a start state that certainly crashes or certainly stops returns 1 or 0 at any resolution. Close behind a stationary lead every start state is of the decided kind, the ego either stops in time or does not. Behind a moving lead the gap closes and reopens with the lead's noise, and the outcome can be probabilistic. The car-following scenarios therefore follow a lead at 10 m/s, under the same lead law as the medium production grid (a lead predicted to hold its speed) started at 10 m/s instead of 0.

A noise-free simulation of the closed loop runs one run of 60 s for each of the 135 combinations of ego speed (12, 14, or 16 m/s) and starting gap (4 to 26 m every 0.5 m) behind a lead held at 10 m/s. Every run settles into steady following and none crashes. Table 8 gives, per ego speed, the closest the ego came to the lead in any run and the gap at which it then follows steadily. Any certified crash probability from these start states is therefore due to the noise, and start states just above the steady-following gap are the ones at which it is fractional.

Ego speed (m/s)	Closest approach over the 45 runs (m)	Steady-following gap (m)
12	3.24	approximately 5.3
14	1.66	approximately 3.6
16	1.12	approximately 3.1

Table 8: The noise-free simulation behind a lead at 10 m/s; each row aggregates the 45 runs at that ego speed.

Table 8 is the noise-free physics, and the car-following scenarios of Table 9 are placed against it. At ego 14 m/s the noise-free ego settles about 3.6 m behind the lead, so the scenarios start at

ego 14 m/s, closing at 4 m/s, at gaps of 5 to 12 m, from just above that settling gap outward. Without noise every one of them settles, so only the noise can cause a crash, and the closer the start the more likely it is to. The medium production grid contains the car-following start states too, but at its resolution it returns only 0 or 1 from them, and the scenarios of Table 6 lie outside the extra-fine window. Every result therefore comes from one named grid, the scenarios of Table 6 from the medium production grid and the car-following scenarios from the extra-fine Following grid.

Scenario	Start state $(g, v_{\text{ego}}, v_{\text{lead}})$
Following Critical	(5, 14, 10)
Following Tight	(6, 14, 10)
Following Near	(8, 14, 10)
Following Mid	(10, 14, 10)
Following Wide	(12, 14, 10)

Table 9: The car-following scenarios on the extra-fine Following grid.

Table 10 reports $P_{\max}$ for the five car-following scenarios at the horizons queried from 10 to 24, every two control periods, the range over which the values move from 0 to 1; Figure 18 draws the same values. At every queried horizon below 10 every scenario returns 0, at horizons 30, 40, 50, and 60 every scenario returns 1, and $P_{\min}$ is 0 for all five scenarios at every horizon queried.

k	Critical (5 m)	Tight (6 m)	Near (8 m)	Mid (10 m)	Wide (12 m)
10	1.859×10^{-3}	1.903×10^{-12}	0	0	0
12	1	0.4891	2.923×10^{-15}	0	0
14	1	1	1.504×10^{-4}	6.379×10^{-22}	0
16	1	1	0.9894	1.835×10^{-5}	4.952×10^{-28}
18	1	1	1	0.2399	7.816×10^{-6}
20	1	1	1	1	0.2771
22	1	1	1	1	0.9998
24	1	1	1	1	1

Table 10: $P_{\max}$ for `F<=k "crash"` across the car-following scenarios on the extra-fine Following grid, rounded to four significant figures. $P_{\min}$ is 0 for all five scenarios at every measured horizon.

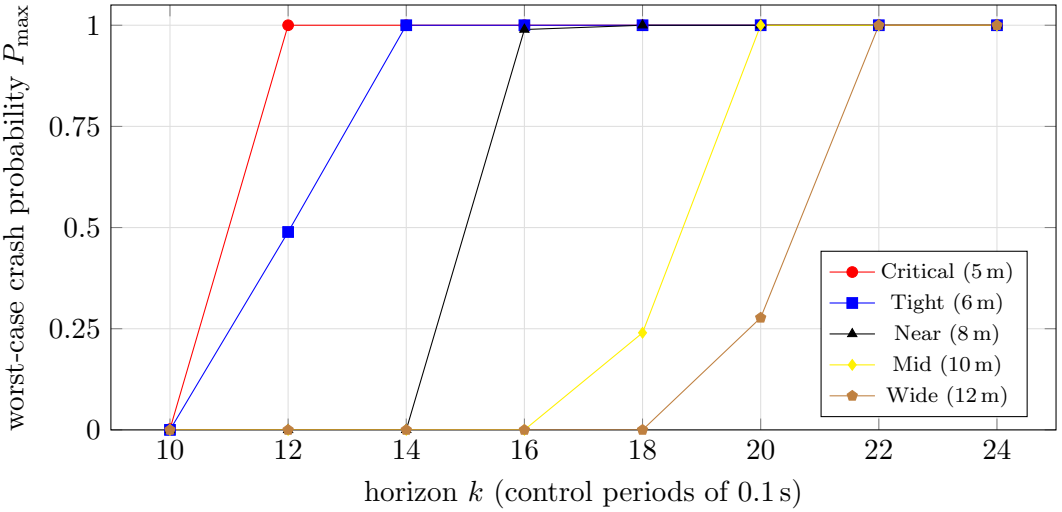


Figure 18: Worst-case crash probability against the horizon for the five car-following scenarios. A larger starting gap delays the rise from 0 to 1.

Every scenario has a fractional $P_{\max}$ at some horizon. At any one horizon the five values are ordered by starting gap, largest at the smallest gap, so the certified worst case reproduces the physical ordering. The horizon at which the worst case becomes certain is

$$k_1 \;=\; \min\{\, k \;:\; P_{\max}(k) = 1 \,\}, \tag{47}$$

where

- $P_{\max}(k)$ is the worst-case crash probability at horizon k;
- k_1 is the smallest queried horizon at which it equals 1, that is, the first horizon at which the worst case is certain to crash.

Table 11 gives k_1 per scenario. A straight line fitted to its five rows has a slope of 1.66 control periods per metre of starting gap; each metre of starting gap defers certainty by 1.66 control periods, that is, 0.60 m of starting gap per control period.

Scenario	Starting gap (m)	k_1, first horizon with $P_{\max} = 1$
Following Critical	5	12
Following Tight	6	14
Following Near	8	18
Following Mid	10	20
Following Wide	12	24

Table 11: The horizon at which the worst case becomes certain, per car-following scenario.

A Monte Carlo simulation of the modelled system, that is, repeated random runs of it drawing the lead-velocity noise that the noise-free simulation switched off, uses the dynamics (37)–(40), the controller (35) as quantised on the grid, and the lead-velocity kernel (42) truncated to the v_{lead} range. It ran 10,000 runs per scenario to horizon 24 under a fixed, recorded random seed; Table 12 gives the crash counts. Every observed crash fraction lies inside its certified bracket, and all lie far below $P_{\max}$, which is the worst case the intervals admit and not the rate at which the modelled system crashes. The two Following Near crashes are consistent with the controller (35), whose first decision from a gap of 8 m is coast while from 5 m it is brake, so Following Near closes at full speed for longer before braking engages.

Scenario	Crashes in 10,000 runs to horizon 24
Following Critical	0
Following Tight	0
Following Near	2
Following Mid	0
Following Wide	0

Table 12: The Monte Carlo simulation, 10,000 runs per scenario.

Cost. Table 13 gives the cost of the queries reported in this subsection.

Queries	Time	Peak memory
Ten-second horizon, medium production grid, $P_{\max}$ and $P_{\min}$	169 min and 99 min	37.5 GB
Car-following scenarios, extra-fine Following grid, $P_{\max}$	58 min	30.9 GB

Table 13: Verification cost on the cluster node.

5.5 Interpretation and outlook

In plain terms, the case study delivers certified worst-case crash probabilities for an AEBS under perfect perception. On the medium production grid the worst case is certified impossible up to a horizon set by the start state and certain from the next queried horizon. On the extra-fine Following grid the worst case is strictly between 0 and 1 at some horizon for every car-following scenario, the values are ordered by starting gap, and the horizon at which the worst case becomes certain grows with the starting gap. Every best-case probability outside the two start states already certain to crash is 0, so each bracket is an upper bound on the crash probability of the modelled system and not an estimate of it. The simulation confirms that the modelled system's crash fractions sit far below the upper bounds.

These results are the perfect-perception baseline of the wider programme (Section 1). The perception layer of Section 5.1 passes the state through unchanged, and every bracket in this section is conditioned on that. The next stage replaces it with measured perception uncertainty from the DNNs, composed with the same certified plant IMDP, and re-runs the same queries; against a perfect-perception baseline the change can only degrade the curves of Figure 18, and the baseline makes the degradation measurable. The baseline is also the reference for correlating DNN verification results with whole-system safety and for extracting the perception requirements that matter most for undegraded safety. Those requirements serve both as specifications a perception component must meet and as runtime monitors that detect safety degradation in operation. The wider framework composes the perception and plant pieces by assume–guarantee contracts, so the brackets certified here are the plant side of that composition.

6 Future work

This paper closes Track 1 of the programme of Section 1. Figure 19 restates the programme with the status of each track. The certified artefacts of Section 5, the two certified configurations and the pipeline that produced them, are the inputs the later tracks build on, and the Following brackets of Section 5.4 are the quantitative baseline against which the perception-aware model's brackets will be compared.

The coming year proceeds in four steps (Figure 20).

1. **Degradation quantification.** A perception component with prescribed error rates is composed with the certified extra-fine configuration, and the Following family is remeasured end-to-end against the baseline. The prescribed rates are state-independent by construction,

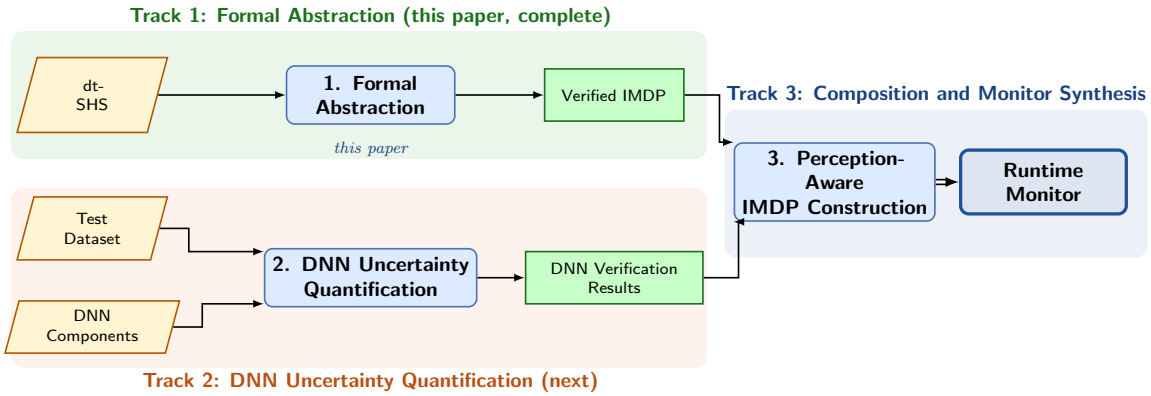


Figure 19: Status of the programme. Track 1 is complete; Tracks 2 and 3 follow the timeline of Figure 20.

so this step reuses the certification machinery of Section 4.3 unchanged and exercises the Track 3 interface early.

2. **Uncertainty quantification.** The prescribed component is replaced by a trained DNN with a labelled test dataset, either adopted from an established driving-perception dataset or generated in simulation, following the practice of [3]. Independent verification techniques partition the test dataset by their joint outcome; one confusion matrix per outcome yields the conditional probability of each perceived state given each true state and outcome, the per-input uncertainty tuples of the programme [3]. We are experimenting with the publicly released DeepDECS implementation, whose augmentation tool composes these probabilities into a discrete-time Markov chain for PRISM; we will adapt it to act on the transition intervals of our Storm-checked IMDP instead, reusing engineering rather than rebuilding it. Measured perception error varies with the state, which breaks the state-independence assumption of Algorithm 2; the interval computation and the containment test must therefore be extended to range over state-dependent noise, and this is the one anticipated change to the pipeline. In the composed model the tuples enter the transition intervals alongside the discretisation over-approximation of this paper, so the interval target of Section 2.3 carries both uncertainty kinds, as it was chosen to do.

3. **Monitor synthesis and validation.** Track 3 composes the tuples with the certified IMDP through the perception-aware construction step of Section 1 and synthesises runtime monitors from the composed model. At run time the monitor reads the verification signals of the perception component, identifies which quantified uncertainty regime the current input falls in, and applies the safety bound computed offline for that regime; when the applicable bound is unacceptable, the monitor overrides the perception output in favour of a verified fallback. The target result is quantitative recovery: verification of the composed system, using the DNN verification results, should recover a substantial part of the degradation measured in the first step.

4. **Monitor extension.** The monitor is extended beyond a single perception channel, with fused sensor inputs as the candidate direction.

Within the present track, two items remain. First, the nondeterministic lead: a lead that chooses per step from a set of admissible accelerations [9] requires a second action axis in the discretiser and the composition, over which the checker would quantify adversarially; it is defined in the implementation but not composed. Second, the width structure of the brackets. The zero lower bounds are exact for width-preserving deterministic dynamics (Section 5.3), so every measured $P_{\min}$ in this paper is 0 and the reported brackets are one-sided: informative upper

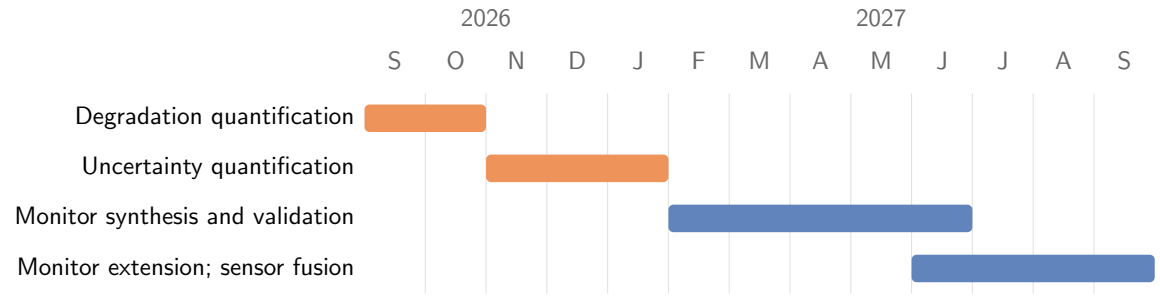


Figure 20: Timeline for Tracks 2 and 3.

bounds, trivial lower bounds. Actuation noise on the ego dynamics would make the ego dimension stochastic and remove the straddling indicators that force the zeros, at the price of a second stochastic dimension in the kernel.

The contribution of this paper to the programme is the certified plant-side component: a sound, automatically constructed IMDP with a discharged containment certificate, together with the pipeline that regenerates it under changed models. The contribution the completed programme aims at is an end-to-end verified learning-enabled AEBS in which every deployed safety claim, including the monitor's switching decisions, traces back to bounds computed offline against a certified model.

7 Conclusion

We have documented an automatic abstraction procedure for discrete-time stochastic hybrid systems whose dynamics mix deterministic and stochastic state variables under a finite action set. The procedure takes such a system and produces a finite interval-valued model of it, together with a certificate that every one-step transition probability of the system lies within the corresponding interval of the model. Under that containment, the probability brackets computed by a robust model checker are sound bounds on the behaviour of the system itself.

We have further documented the application of the procedure to an autonomous emergency braking system, a representative system of this class. The certified models answer the safety question posed of them: from the most dangerous starting conditions a crash is certified as unavoidable, from safer starting conditions the horizon within which safety is decidable is located, and at finer resolution the models discriminate between starting conditions by degree of danger rather than by a coarse safe-or-unsafe verdict. All reported brackets carry informative upper bounds and trivial lower bounds; Section 6 states how the programme builds on these results.

References

[1] Alessandro Abate, Alessandro D'Innocenzo, and Maria Domenica Di Benedetto. Approximate abstractions of stochastic hybrid systems. *IEEE Transactions on Automatic Control*, 56(11):2688–2694, 2011.

[2] Alessandro Abate, Maria Prandini, John Lygeros, and Shankar Sastry. Probabilistic reachability and safety for controlled discrete time stochastic hybrid systems. *Automatica*, 44(11):2724–2734, 2008.

[3] Radu Calinescu, Calum Imrie, Ravi Mangal, Genaína Nunes Rodrigues, Corina Păsăreanu, Misael Alpizar Santana, and Gricel Vázquez. Controller synthesis for autonomous systems with deep-learning perception components. *IEEE Transactions on Software Engineering*, 50(6):1374–1395, 2024.

[4] Samuel Coogan and Murat Arcak. Efficient finite abstraction of mixed monotone systems. In *Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control (HSCC)*, pages 58–67. ACM, 2015.

[5] Christian Dehnert, Sebastian Junges, Joost-Pieter Katoen, and Matthias Volk. A Storm is coming: A modern probabilistic model checker. In *Computer Aided Verification (CAV)*, volume 10427 of *Lecture Notes in Computer Science*, pages 592–600. Springer, 2017.

[6] Alessandro D'Innocenzo, Alessandro Abate, and Joost-Pieter Katoen. Robust PCTL model checking. In *Proceedings of the 15th ACM International Conference on Hybrid Systems: Computation and Control (HSCC)*, pages 275–285. ACM, 2012.

[7] Tommaso Dreossi, Alexandre Donzé, and Sanjit A. Seshia. Compositional falsification of cyber-physical systems with machine learning components, 2018.

[8] Sadegh Esmaeil Zadeh Soudjani and Alessandro Abate. Adaptive and sequential gridding procedures for the abstraction and verification of stochastic processes. *SIAM Journal on Applied Dynamical Systems*, 12(2):921–956, 2013.

[9] European New Car Assessment Programme. Test protocol – AEB car-to-car systems. Test Protocol Version 3.0.3, Euro NCAP, 2021.

[10] Robert Givan, Sonia Leach, and Thomas Dean. Bounded-parameter Markov decision processes. *Artificial Intelligence*, 122(1–2):71–109, 2000.

[11] Darald J. Hartfiel. *Markov Set-Chains*, volume 1695 of *Lecture Notes in Mathematics*. Springer, 1998.

[12] Marta Kwiatkowska, Gethin Norman, and David Parker. Probabilistic model checking: Applications and trends, 2025.

[13] Abolfazl Lavaei, Sadegh Soudjani, Alessandro Abate, and Majid Zamani. Automated verification and synthesis of stochastic hybrid systems: A survey. *Automatica*, 146:110617, 2022.

[14] Corina Pasareanu, Ravi Mangal, Divya Gopinath, and Huafeng Yu. Assumption generation for the verification of learning-enabled autonomous systems, 2023.

[15] Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. Towards verified artificial intelligence, 2020.

[16] D. Seto, B. Krogh, L. Sha, and A. Chutinan. The simplex architecture for safe online control system upgrades. In *Proceedings of the 1998 American Control Conference. ACC (IEEE Cat. No.98CH36207)*, volume 6, pages 3504–3508 vol.6, 1998.